



软件工程概论

Cynthia

Table of Contents

1. 软件与软件工程介绍	4
1.1. 软件的概念、特性和分类	4
1.2. 软件危机与软件工程	4
1.3. 软件工程	5
1.3.1. 软件工程的目标	5
1.4. 软件生存期	6
1.4.1. 软件生存期三阶段	6
1.4.2. 典型文档	7
1.5. 软件工程方法概述	7
1.5.1. 概念	7
1.5.2. 软件工程三要素	7
1.5.3. 三个软件工程方法学	7
1.6. 软件工具的概述	8
1.6.1. 软件工具的概念	8
1.6.2. 软件工具的分类	8
1.6.3. 常用软件工具	8
1.7. 小结	9
2. 软件生存期模型	10
2.1. 瀑布模型	10
2.1.1. 瀑布模型特点:	10
2.1.2. 瀑布模型的优点:	10
2.1.3. 瀑布模型的缺点:	11
2.2. 快速原型模型	11
2.2.1. 快速原型模型的优点	11
2.2.2. 快速原型模型的缺点	11
2.3. 增量模型	12
2.3.1. 增量模型的优点	12
2.3.2. 增量模型的缺点	12
2.4. 螺旋模型	13
2.4.1. 螺旋模型的优点	14
2.4.2. 螺旋模型的缺点	14
2.5. 喷泉模型	14
2.6. 统一过程	14
2.7. 基于构件的开发模型	15
2.8. 敏捷过程	15
2.8.1. 极限编程 XP	16
2.8.2. Scrum	16
2.9. 小结	17
3. 软件需求获取与结构化分析方法	18
3.1. 需求获取与需求分析阶段的任务	18

3.1.1.	需求获取的任务和原则	18
3.1.2.	需求获取的过程	18
3.1.3.	软件需求分析阶段的任务	19
3.2.	结构化分析方法 SA	19
3.2.1.	结构化分析模型	19
3.2.2.	功能建模	20
3.2.3.	数据建模	22
3.2.4.	行为建模	22
3.2.5.	数据字典	23
3.2.6.	加工规格说明	24
3.3.	小结	24
3.4.	系统需求规格说明	25
4.	结构化设计方法	26
4.1.	软件设计的概念及原则	26
4.1.1.	耦合性与模块独立性	26
4.1.2.	内聚性与模块独立性	27
4.2.	结构化设计	28
4.2.1.	软件设计的任务	28
4.2.2.	模块结构及其表示	28
4.3.	体系结构设计	29
4.3.1.	基于数据流方法的设计过程	29
4.3.2.	典型的数据流类型和系统结构	30
4.3.3.	变换型映射方法	31
4.3.4.	事务型映射方法	32
4.3.5.	软件模块结构的改进方法	33
4.4.	接口设计	33
4.5.	数据设计	34
4.5.1.	文件设计	34
4.5.2.	数据库设计	34
4.6.	过程设计	34
4.6.1.	结构化程序设计	34
4.7.	小结	35
5.	面向对象方法与 UML	36
5.1.	面向对象的概念与开发方法	36
5.2.	UML 简介	37
5.3.	UML 的事物	38
5.4.	UML 的关系	38
5.4.1.	依赖关系	38
5.4.2.	关联关系	38
5.4.3.	泛化关系	40
5.4.4.	实现关系	40
5.5.	UML 的图	41

5.5.1.	用例图	41
5.5.2.	类图	43
5.5.3.	顺序图	44
5.6.	状态图	48
5.6.1.	活动图	49
5.6.2.	构件图与部署图	52
5.6.3.	部署图	54
5.7.	UML 的 4+1 视图模型	54
5.8.	小结	55
6.	面向对象分析	57
6.1.	面向对象分析概述	57
6.1.1.	分析模型	57
6.1.2.	需求分析	57
6.2.	基于场景建模	57
6.3.	基于类的建模	58
6.4.	基于行为的建模	58
7.	软件体系结构与设计模式	59
7.1.	软件体系结构的基本概念	59
8.	面向对象设计	60
8.1.	面向对象设计过程与准则	60
8.2.	体系结构模块及依赖性	60
8.2.1.	类及其依赖性	60
8.2.2.	接口及其依赖性	61
8.2.3.	包及其依赖性	61
8.2.4.	构件及其依赖性	61
9.	软件测试	62
9.1.	单元测试	62
9.2.	集成	62
9.2.1.	冒烟测试	62
9.2.2.	回归测试	63
9.2.3.	面向对象的集成测试	63
9.2.4.	确认测试	63
9.3.	系统测试	63
9.4.	调试	63
10.	软件测试用例设计	64
10.1.	黑盒测试（功能测试）	64
10.1.1.	黑盒测试用例设计方法	64
10.2.	白盒测试	64
10.2.1.	分支结构	65
10.2.2.	循环结构	65

10.2.3. 基本路径测试	65
11. 软件项目管理	67
11.1. 进度管理	67
11.1.1. 甘特图	67
11.1.2. 时标网状图	68
11.2. 配置管理	68
11.2.1. 配置管理概述	68
11.2.2. 软件配置标识	68
11.2.3. 变更管理	68
11.2.4. 版本控制	69
11.3. 质量管理	69
11.3.1. 正确理解和正确对待软件质量问题	69
11.3.2. 质量管理措施	69
11.4. 小结	69

Chapter 1: 软件与软件工程介绍

1.1. 软件的概念、特性和分类

1. 概念

软件 是一系列按照特定顺序组织的计算机数据和指令的集合，包括程序、数据和文档。

- ①. **程序**是按事先设计的功能和性能要求执行的**指令序列**
- ②. **数据**是使程序能正常操纵信息的数据**结构**
- ③. **文档**是与程序开发，维护和使用有关的**图文材料**

2. 软件的特性

- ①. 形态特性：无形的逻辑实体，无法用常规物理指标度量。
- ②. 智能特性：凝聚脑力劳动，体现知识与智慧，可解决复杂问题。
- ③. 开发特性：依赖个体劳动，尚未完全自动化。
- ④. 质量特性：难以实现完全无缺陷。
- ⑤. 生产特性：复制成本低，与硬件生产差异显著。
- ⑥. 管理特性：开发管理需适配软件独特性。
- ⑦. 环境特性：依赖特定软硬件运行环境。
- ⑧. 维护特性：维护逻辑与传统产品不同。
- ⑨. 废弃特性：非“用坏”而是因需求变更或技术迭代被淘汰。
- ⑩. 应用特性：渗透国民经济和国防各领域，是核心支撑。

3. 软件分类

- ①. 系统软件：操作系统、数据库管理系统、设备驱动程序、网络处理程序等。
- ②. 支撑软件（工具软件）：纵向（分析、设计、编码、测试工具）、横向（项目管理、配置管理工具）。
- ③. 应用软件：工程与科学计算软件、ERP 软件、CAD/CAM 软件、办公自动化软件等。
- ④. 可复用软件：标准函数库、类库、构件库等。

1.2. 软件危机与软件工程

1. 软件危机

- 爆发背景：20 世纪 60 年代末，软件发展滞后于硬件，无法满足社会需求。
- 主要表现表现：软件的发展速度远远**滞后于硬件的发展速度**，不能满足社会日益增长的软件需求。软件**开发周期长、成本高、质量差、维护困难**。
- 典型表现：
 - ①. 对软件**开发成本和进度的估计常常很不准确**。
 - ②. 用户对“已完成的”软件系统不满意的现象经常发生。

- ③. 软件产品的质量往往靠不住。
- ④. 软件常常是不可维护的。
- ⑤. 软件通常没有适当的文档资料。
- ⑥. 软件成本在计算机系统总成本中所占的比例逐年上升。
- ⑦. 软件开发生产率提高的速度，既跟不上硬件的发展速度，也远远跟不上计算机应用迅速普及深入的趋势。
- 核心发生原因：缺乏开发经验与数据积累、需求沟通障碍、开发过程不规范、系统复杂性高、缺乏有效质量评测手段。
- 典型案例：1996 年阿丽亚娜 5 型火箭爆炸（代码重用适配错误）、1999 年火星气候卫星坠毁（单位换算错误 + 沟通疏漏）。

1.3. 软件工程

1. 软件工程

- 背景：为了克服软件危机，1968 年 10 月在北大西洋公约组织（NATO）召开的计算机科学会议上，首次提出“软件工程”的概念，试图将工程化方法应用于软件开发。
- 定义：软件工程是指导计算机软件开发和维护的工程学科，结合工程原理、管理技术与最佳技术方法，实现软件的经济开发与有效维护。
- 基本目标：降低软件低开发成本、满足功能需求、良好性能、易移植、低维护费用、按时交付。
- **软件的质量特性(6)**：功能性、可靠性、易用性、效率、可维护性和可移植性。
 - **功能性**是指软件所实现的功能达到它的设计规范和满足用户需求的程度，其质量子特征包括适合性、正确性、安全保密性；
 - **可靠性**是指在规定的时间和条件下，软件能够正常维持其工作的能力，其质量子特征包括成熟性、恢复性、容错性；
 - **易用性**是指为了使用该软件所需要的能力，其质量子特征包括易理解性、易学习性、易操作性；
 - **效率**是指规定的条件下用软件实现某种功能所需要的计算机资源的有效性，其质量子特征包括时间特性、资源特性；
 - **可维护性**是指当环境改变或软件运行发生故障时，为使其恢复正常运行所做努力的程度，其质量子特征包括易分析性、易修改性、易测试性；
 - **可移植性**是指软件从某一环境转移到另一环境时所做努力的程度，其质量子特征包括适应性、易替换性。

1.3.1. 软件工程的目标

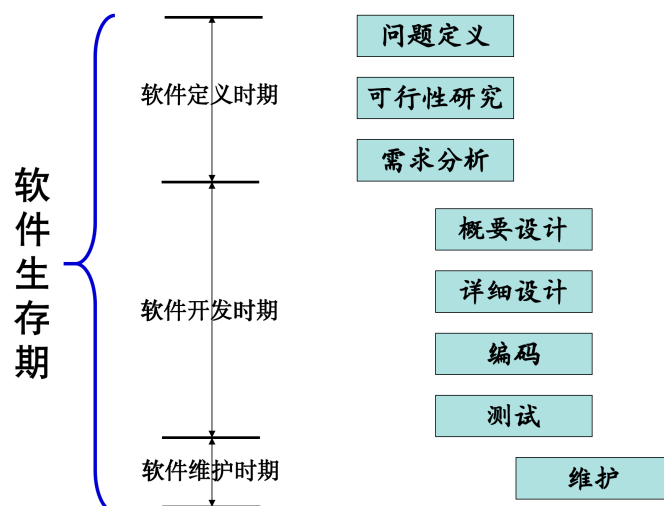
1. 转变对软件的认识（程序 → 系统）
2. 转变思维定式（程序员 → 系统分析师）

软件工程的目的是运用先进的软件开发技术和管理方法来提高软件的质量和生产率，也就是以较短的周期、较低的成本生产出高质量的软件产品，并最终实现软件的工业化生产。

1.4. 软件生存期

软件也有一个孕育、诞生、成长、成熟和衰亡的生存过程，我们称这个过程为**软件生命周期**或**软件生存期**。

1.4.1. 软件生存期三阶段



1. 软件的生命周期：

①. 软件定义时期 (3)：

- 问题定义 (明确问题性质与目标)
 - 提交**问题性质、工程目标与规模报告**。
- 可行性研究 (从技术 / 经济 / 社会研究各方案的可行性)
 - 提交**可行性研究报告**。
- 需求分析 (对用户的需求进行分析和详细定义、定义系统功能与约束)。
 - 提交**软件需求说明书、初步用户手册**。

②. 软件开发时期 (4)：

- 概要设计 (确定系统架构与模块关系，提交的文档是软件的**概要设计说明书**)
- 详细设计 (确定实现模块功能所需要的算法和数据结构，提交的文档是软件的**详细设计说明书**)
- 编码与单元测试
 - 写出正确的容易理解、容易维护的程序模块。
 - 提交的文档为**源程序、详尽的程序说明和单元测试报告**。
- 集成与系统测试
 - 通过各种类型的测试(及相应的调试)使软件达到预定的要求。

- 提交的文档为**测试计划**、**详细测试方案**以及**实际测试结果**等。

③. 软件运行维护时期 (1):

- 提交**维护记录与报告**。
- 4 类维护活动
 - 改正性维护 (修复错误) **【主要任务, 占比最大】**
 - 适应性维护 (适配环境变化)
 - 完善性维护 (扩充功能) **【主要任务】**
 - 预防性维护 (即修改软件为将来的维护活动预先做准备)

1.4.2. 典型文档

开发过程中的典型文档

1. 软件需求规格说明书: 描述将要开发的软件做什么。
2. 项目计划: 描述将要完成的**任务及其顺序**, 并估计所需要的时间及工作量。
3. 软件测试计划: 描述如何测试软件, 使之确保软件应实现规定的功能, 并达到预期的性能。
4. 软件设计说明书: 描述软件的结构, 包括概要设计及详细设计。
5. 用户手册: 描述如何使用软件。

1.5. 软件工程方法概述

1.5.1. 概念

- 软件工程包含**技术**和**管理**两方面的内容, 是技术和管理紧密结合所形成的工程学科。
- 通常将软件开发全过程中使用的一整套**技术方法的集合称为方法学**, 也称为**范型**。
- 目前使用最广泛的软件工程方法学: 传统方法学 (结构化方法学, 也称生命周期方法学), 面向对象方法学。

1.5.2. 软件工程三要素

三要素: 方法、工具和过程。

1. 软件工程方法为软件开发提供了“如何做”的**技术**
2. 软件工具为软件工程方法提供了自动的或半自动的**软件支撑环境**
3. 过程是为了获得高质量的软件所需要完成的一系列**任务框架**, 它规定了完成各项任务的工作步骤。

1.5.3. 三个软件工程方法学

1. 结构化方法

将软件生命周期的全过程依次划分为若干个阶段, 采用结构化技术来完成每个阶段的任务。

- 强调**自顶向下顺序**地完成软件开发的各阶段任务;
- **要么面向行为，要么面向数据**，缺乏使两者有机结合的机制。

2. 面向对象方法学

- 结合数据和对数据的操作。
- 软件开发过程是多次反复迭代的演化过程。
- 在概念和表示方法上的一致性，保证了各项开发活动之间的平滑过渡。
- 适用于大型、复杂及交互性比较强的系统。

3. 面向服务方法

服务是指与业务相关且独立于技术的业务接口。面向服务方法通过组合各种服务来构建软件系统。

- 关键特性：松耦合、抽象性、粗粒度、可复用性。
- 以业务为中心；
- 灵活适应变化；
- 重用已有资源，提升开发效率

4. 面向数据方法

面向数据的方法是基于数据思维，从业务逻辑的角度，强调一切业务“数据化”；从体系结构的角度，突出“面向数据和以数据为核心”的思想。

5. 形式化方法

形式化方法是一种基于形式化数学变换的软件开发方法，它可将系统的规格说明转换为可执行的程序。

- 软件需求规格说明被细化为用数学记号表达的详细的形式化规格说明；
- 设计、实现和单元测试等开发过程由一个变换开发过程代替。通过一系列变换将形式化规格说明细化成为程序。

1.6. 软件工具的概述

1.6.1. 软件工具的概念

软件工具是指能支持软件生存周期中某一阶段（如系统定义、需求分析、设计、编码、测试或维护等）的需要而使用的软件工具。

1.6.2. 软件工具的分类

1. 支持软件**开发**过程的工具：主要有需求分析工具、设计工具、编码工具、排错工具、测试工具等。
2. 支持软件**维护**过程的工具：主要有版本控制工具、文档分析工具、信息库开发工具、逆向工程工具、再工程工具等。
3. 支持软件**管理**过程和**支持**过程的工具：主要有项目管理工具、配置管理工具、软件评价工具等。

1.6.3. 常用软件工具

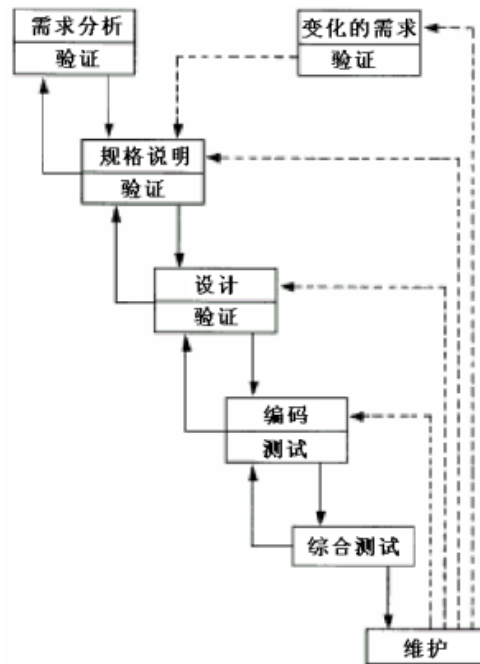
需求分析与设计工具、编码工具与排错工具、测试工具（单元、组装、系统）

1.7. 小结

- 1.什么是软件？是一系列按照特定顺序组织的计算机数据和指令的集合，包括程序、数据和文档。
- 2.什么是软件危机，其内容主要是指什么？
- 3.什么是软件工程？
- 4.软件工程的目标及其组成部分。方法、工具和过程。
- 5.软件开发方法的定义。
- 6.好的软件的一些主要衡量指标。

Chapter 2: 软件生存期模型

2.1. 瀑布模型



2.1.1. 瀑布模型特点:

1. 阶段间具有顺序性和依赖性

- ①. 必须等前一阶段的工作完成之后，才能开始后一阶段的工作；
- ②. 前一阶段的输出文档就是后一阶段的输入文档。

2. 推迟实现的观点

- ①. 编码之前设置了系统分析和系统设计的各个阶段，这两个阶段主要考虑目标系统的逻辑模型，不涉及软件的物理实现。
- ②. 清楚地区分逻辑设计与物理设计，尽可能推迟程序的物理实现。

3. 质量保证的观点

- ①. 每个阶段都必须完成规定的文档，没有交出合格的文档就是没有完成该阶段的任务。
- ②. 每个阶段结束前都要对所完成的文档进行评审，以便尽早发现问题，改正错误。

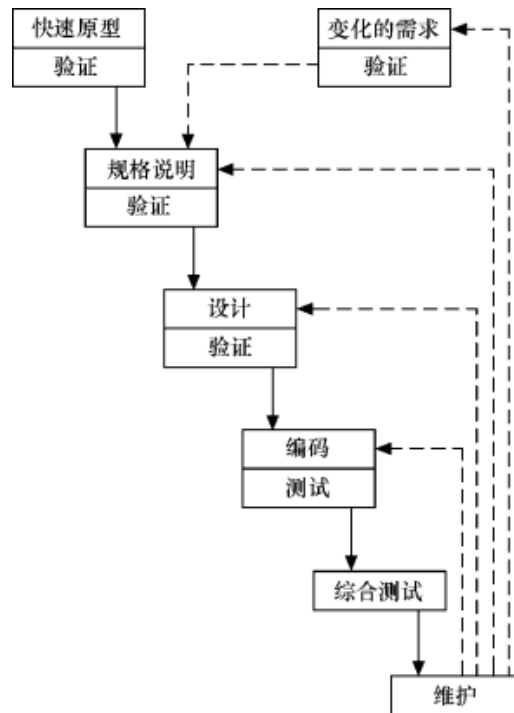
2.1.2. 瀑布模型的优点:

1. 可强迫开发人员采用规范化的方法。
2. 严格地规定了每个阶段必须提交的文档。
3. 要求每个阶段交出的所有产品都必须是经过验证的。

2.1.3. 瀑布模型的缺点:

1. 灵活性差: 需求变更需回溯早期阶段, 修改成本高、周期长;
3. 风险滞后: 测试阶段才暴露问题 (如架构缺陷), 修复代价大;
4. 工作量大: 阶段间产生大量文档, 增加管理与维护成本。
5. 瀑布模型只适用于项目开始时需求已确定的情况。

2.2. 快速原型模型



快速原型是快速建立起来的可以在计算机上运行的程序，它所能完成的功能往往是最终产品能完成的功能的一个子集。

2.2.1. 快速原型模型的优点

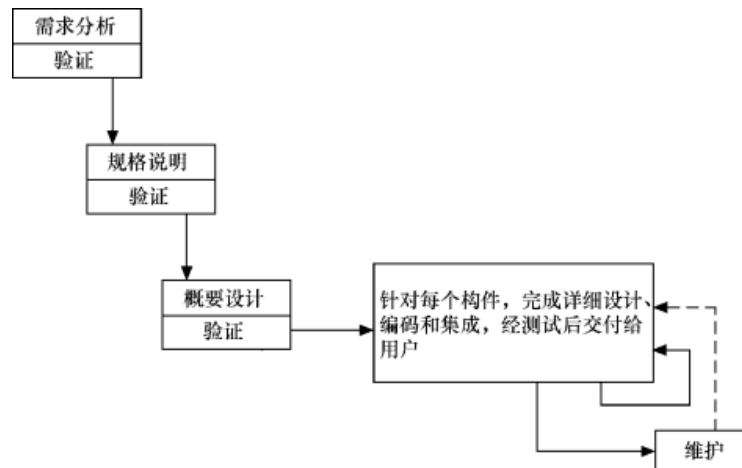
1. 有助于满足用户的真实需求。
2. 原型系统已经通过与用户的交互而得到验证，因此规格说明能够正确描述用户需求。
3. 软件开发基本按照线性顺序进行。
4. 在开发过程的后续阶段不会因为发现规格说明文档的错误而进行较大的返工。
5. 开发人员通过建立原型系统已经学到了许多东西，因此，在设计和编码阶段发生错误的可能性也比较小。
6. 快速（突出特点）。开发人员应该尽可能快地建造出原型系统，以加速软件开发过程，节约软件开发成本。

2.2.2. 快速原型模型的缺点

1. 为了使原型尽快的工作，没有考虑软件的总体质量和长期的可维护性。

2. 为了演示，可能采用不合适的操作系统、编程语言、效率低的算法，这些不理想的选择成了系统的组成部分。

2.3. 增量模型



1. 使用增量模型开发软件时，把软件产品作为一系列的增量构件来设计、编码、集成和测试。
2. 每个构件由多个相互作用的模块构成，并且能够完成特定的功能。

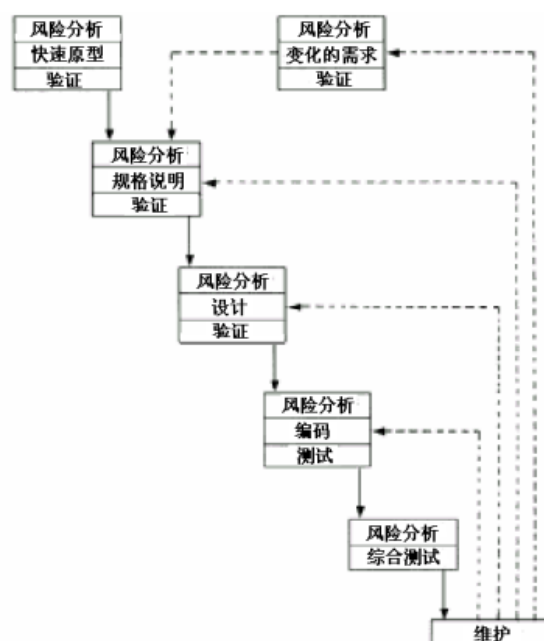
2.3.1. 增量模型的优点

1. 能在**较短时间内向用户提交可完成一些有用工作的产品**，即从第1个构件交付之日起，用户就能做一些有用的工作。
2. 逐步增加产品的功能可以**使用户有较充裕的时间学习和适应新产品**，从而减少一个全新的软件可能给用户组织带来的冲击。
3. **项目失败的风险较低**，虽然在某些增量构件中可能遇到一些问题，但其他增量构件将能够成功地交付给客户。
4. **优先级最高的服务首先交付**，然后再将其他增量构件逐次集成进来。因此，最重要的系统服务将接受最多的测试。

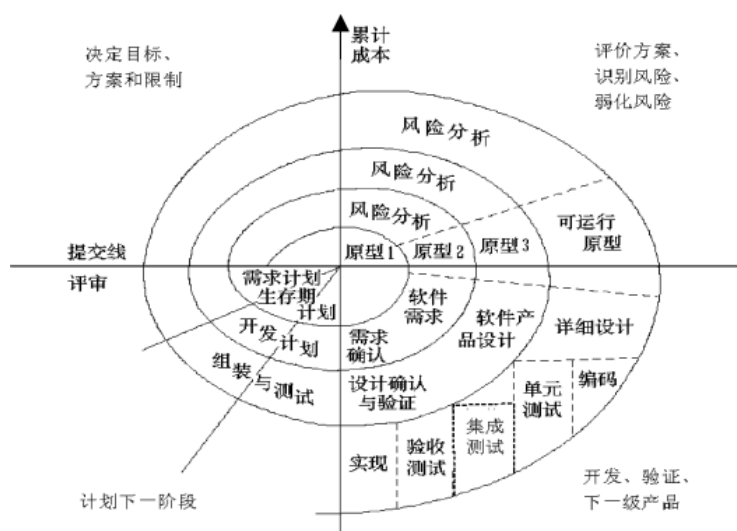
2.3.2. 增量模型的缺点

1. 在把每个新的增量构件集成到现有软件体系结构中时，**必须不破坏原来已经开发出的产品**。
2. **软件体系结构必须是开放的**，即向现有产品中加入新构件的过程必须简单、方便。

2.4. 螺旋模型



可视为看做在每个阶段之前都增加了**风险分析过程**的**快速原型模型**。



在螺旋线上的每一个循环表示过程的一个阶段。

每个阶段开始时的任务是确定该阶段的目标、为完成这些目标选择方案及设定这些方案的约束条件。接下来的任务是，从风险角度分析上一步的工作结果，努力排除各种潜在的风险，通常用建造原型的方法来排除风险。如果成功地排除了所有风险，则启动下一步开发步骤，在这个步骤的工作过程相当于纯粹的瀑布模型。最后是评价该阶段的工作成果并计划下一个阶段的工作。

4个象限的4个活动：

1. 目标设定
2. 风险估计与弱化
3. 开发与验证

4. 计划

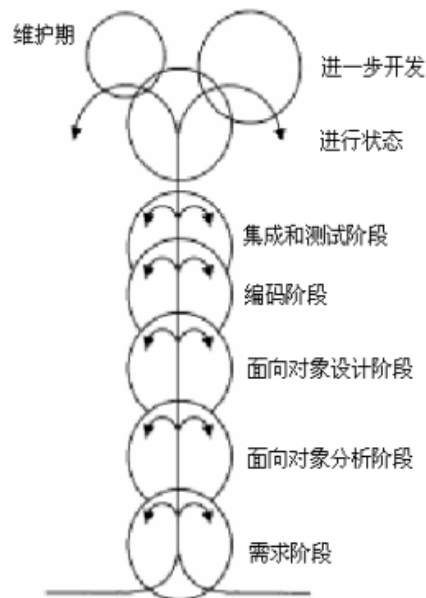
2.4.1. 螺旋模型的优点

1. 对可选方案和约束条件的强调有利于已有软件的重用，也有助于把软件质量作为软件开发的一个重要目标。
2. 减少了过多测试或测试不足所带来的风险。
3. 在螺旋模型中维护只是模型的另一个周期，因而在维护和开发之间并没有本质区别。

2.4.2. 螺旋模型的缺点

螺旋模型是风险驱动的，因此要求软件开发人员必须具有丰富的风险评估经验和这方面的专门知识，否则将出现真正的风险

2.5. 喷泉模型



“喷泉”一词体现了迭代和无间隙特性。图中代表不同阶段的圆圈相互重叠，这明确表示两个活动之间存在重叠。循环迭代。

适合面向对象开发。

2.6. 统一过程

6 个核心工作流

1. 业务建模工作流

用商业用例为商业过程建立文档

2. 需求工作流

目标是描述系统应该做什么，确保开发人员构建正确的系统。为此，需明确系统的功能需求和非功能需求（约束）。

3. 分析和设计 workflow

其目标是说明如何做。结果是分析模型和设计模型

4. 实现 workflow

用分层的方式组织代码的结构，用构件的形式来实现类，对构件进行单元测试，将构件集成到可执行的系统中

5. 测试 workflow

验证对象之间的交互、是否所有的构件都集成了、是否正确实现了所有需求、查错并改正

6. 部署 workflow

制作软件的外部版本、软件打包、分发、为用户提供帮助和支持

4 个阶段

1. 初始阶段

初始阶段主要关注项目计划和风险评估，其目的是确定是否值得开发目标信息系统。

2. 细化阶段

细化阶段关心定义系统的总体框架，其目标是：细化初始需求、体系结构、风险优先级、业务案例以及制订项目管理计划。

3. 构造阶段。构造阶段是建立系统

4. 移交阶段

以发布完整的系统而终止，其目标是确保信息系统真正满足客户的需求。

2.7. 基于构件的开发模型

基于构件的软件工程是强调使用可复用的软件“构件”来设计和构造基于计算机的系统的过程。

考虑的焦点是“集成”，而不再是“实现”。

2.8. 敏捷过程

原则：

1. 我们最优先要做的是通过尽早、持续交付有价值的软件来使客户满意。
2. 即使在开发的后期，也欢迎需求变更。敏捷过程利用变更为客户创造竞争优势。
3. **经常交付可运行软件**，交付的间隔可以从几个星期到几个月，交付的时间间隔越短越好。
4. 在整个项目开发期间，业务人员和开发人员必须天天都在一起工作。
5. 围绕有积极性的个人构建项目。给他们提供所需的环境和支持，并且信任他们能够完成工作.....

三个假设识别敏感项目：

1. 提前预测需求的稳定与会变化性非常困难。同样的，预测项目进行客户优先级的变化也很困难。

2. 对很多软件，设计和构建是交错进行的。事实上，两种活动应当顺序开展以保证通过构建实施来验证设计模型，而在通过构建验证之前很难估计应该设计到什么程度。
3. 从制订计划的角度来看，分析、设计、构建和测试并不像我们所设想的那么容易预测。

特点：

1. 迭代式交付：以 2-4 周为迭代周期，持续交付可运行软件。
2. 用户紧密参与：客户全程参与迭代评审，提供实时反馈。
3. 轻量文档：聚焦“必要文档”，优先保证可工作软件。
4. 自组织团队：团队自主分配任务、决策，强调协作和沟通。
5. 适应变化：将需求变更视为竞争优势，快速调整迭代计划。

2.8.1. 极限编程 XP

XP 包含了策划、设计、编码和测试 4 个框架活动的规则和实践。

1. 策划
 - 开始于建立“用户故事”
 - 敏捷团队评估每一个故事并给出成本
2. 设计：使用简单设计而不是更为复杂的表述
3. 编码
 - 建议在开始编码之前为每一个故事开发一系列单元测试
 - 鼓励结对编码
4. 测试：
 - 所有的单元测试每天都要执行
 - “验收测试”由客户定义，将着眼于客户可见的、可评审的系统级的特征和功能。

2.8.2. Scrum

Scrum 是一种迭代式增量开发过程

角色：

1. Scrum Master，确保团队合理运作 Scrum
2. 产品负责人，代表利益相关者的意愿，编写用户故事，确定优先级，并放入产品订单
3. 开发团队，负责交付产品的团队，通常由具有跨职能技能的人组成

关键词条：

1. 产品待定项
 - 量化的用户需求，条目化地表达实际需要开发的需求

2. 冲刺待项

当前迭代的细化任务

2.9. 小结

- 1.什么是软件生命周期？主要分为哪些阶段？
各个阶段的主要任务及产生的主要制品？
- 2.典型的软件开发过程模型的特点（优缺点）
及要求，特别是原型法、瀑布模型、螺旋模型、增量和迭代等。
- 3.敏捷开发方法和极限编程、scrum的特点。

模型 / 方法	核心关键词	优势领域	主要局限
瀑布模型	线性、文档驱动	需求稳定、重视合规文档	不适应变化、反馈滞后
快速原型模型	可视化、用户验证	需求模糊、交互类软件	忽视长期质量、工具依赖强
螺旋模型	风险驱动、迭代	大型复杂、高风险项目	周期长、资源要求高
增量模型	分批次交付、架构开放	核心需求明确、需快速上线	架构设计要求高、沟通成本高
敏捷开发（XP/Scrum）	快速迭代、客户协作	需求频繁变更、互联网产品	团队要求高、文档不足

Chapter 3: 软件需求获取与结构化分析方法

3.1. 需求获取与需求分析阶段的任务

3.1.1. 需求获取的任务和原则

需求获取的**主要任务**是与客户或用户沟通，了解系统或产品的目标是什么？客户或用户想要实现什么？系统和产品如何满足业务的要求，最终系统或产品如何用于日常工作？

获取并理解用户的需求是软件工程师所面对的最困难的任务之一。

导出需求困难的原因：

1. 系统的目标或范围问题；
2. 需求不准确性问题；
3. 需求的易变问题；

需求获取的任务

1. 发现和分析问题，并分析问题的原因/结果关系。
2. 与用户进行各种方式的交流，并使用调查研究方法收集信息。
3. 按照三个成分观察问题的不同侧面：即数据、过程和接口。
4. 将获取的需求文档化，形式有用例、决策表、需求表等。

需求获取应遵循的原则

1. 深入浅出的原则。需求获取要尽可能全面、细致。获取的需求是个全集，目标系统真正实现的是个子集。
2. 以流程为主线的原则。在与用户交流的过程中，应该用流程将所有的内容串起来。

3.1.2. 需求获取的过程

1. 开发高层的业务模型
2. 定义项目范围和高层需求
3. 识别用户类和用户代表
4. 获取具体的需求

确定了项目范围和高层需求，并确定了用户类及用户代表后，就需要获取更具体、完整和详细的需求。

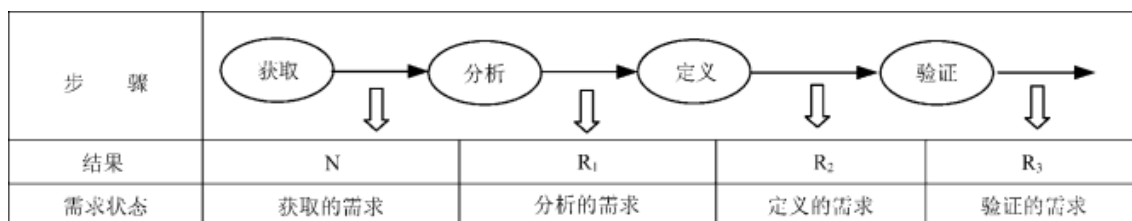
5. 确定目标系统的业务 workflow

具体到当前待开发的应用系统，确定系统的业务 workflow 和主要的业务规则，采取需求调研的方法获取所需的信息。

6. 需求整理与总结

3.1.3. 软件需求分析阶段的任务

可以把软件需求分析阶段的工作分为 4 个步骤，即获取需求、分析需求、定义需求和验证需求。



1. 需求获取

通过启发、引导从客户（或用户）那里得到的**原始需求**是他们的业务要求 (needs)，简称为 N 。

2. 需求分析

- ①. **完整性**：每项获取的需求都应给出清楚的描述，使得软件开发工作能够取得设计和实现该功能所需要的全部必要信息。
- ②. **正确性**：获取的每项需求必须是准确无误的，需求描述无歧义性。
- ③. **合理性**：各项需求之间、软件需求与系统需求之间应是协调一致的，不应存在矛盾和冲突。
- ④. **可行性**：包括技术可行性、经济可行性、社会可行性。
- ⑤. **充分性**：获取的需求是否全面、周到。

得到 $R_1 = (b + c) = (N - a) + c$ （分析去掉 a ，加上 c ）

3. 需求定义

将已经过分析的需求清晰、全面、系统、准确地**描述成为正式的文档**，这一步定义需求的工作就是**编写需求规格说明**。

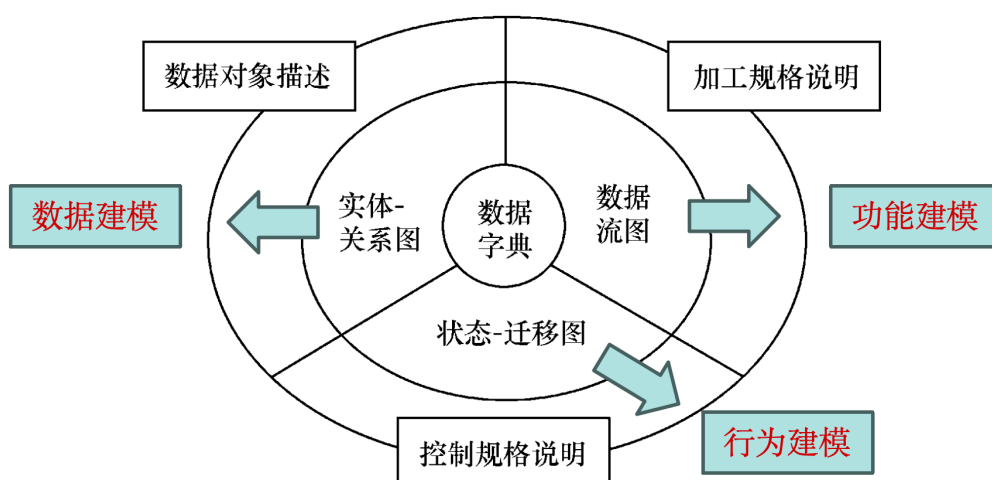
4. 需求验证

为了确保需求规格说明准确无误，并能为客户理解和接受，需要对其进行严格的评审。

3.2. 结构化分析方法 SA

3.2.1. 结构化分析模型

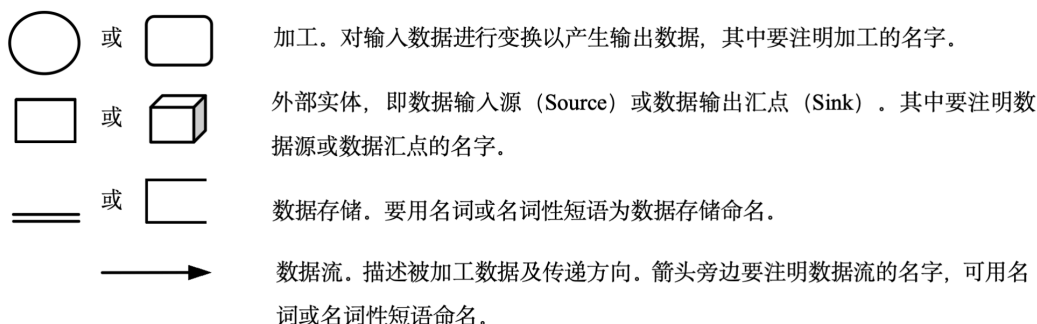
结构化分析方法是一种建模技术。



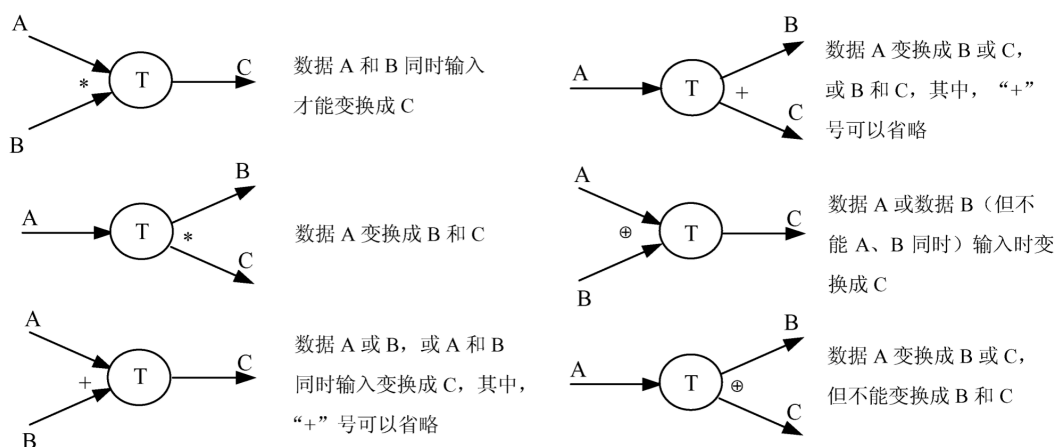
3.2.2. 功能建模

1. 概念

功能建模的思想就是用**抽象模型**的概念，按照软件内部数据传递、变换的关系，自顶向下逐层分解，直到找到满足功能要求的所有可实现的软件为止。功能模型用**数据流图（DFD）**来描述。



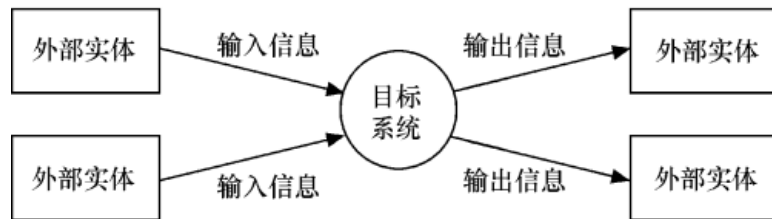
2. 多个数据流的关系



3. 环境图（顶层数据流图）

仅包括一个数据处理过程，也就是要开发的目标系统。

- 作用：确定系统在其环境中的位置，通过确定系统的输入和输出与外部实体的关系确定其边界。



• 招生系统需求描述

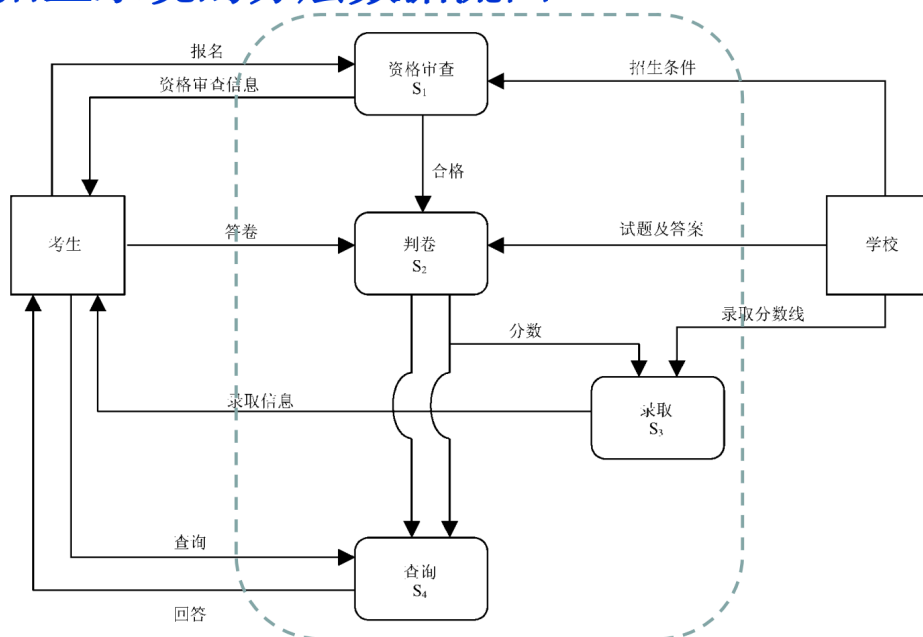
- 学校首先公布招生条件，考生根据自己的条件报名，之后系统进行资格审查，并给出资格审查信息；
- 对于资格审查合格的考生可以参加答卷，系统根据学校提供的试题及答案进行自动判卷，并给出分数及答题信息，供考生查询；
- 最后系统根据学校的录取分数线进行录取，并将录取信息发送给考生。



4. 分层数据流图

- 顶层数据流图
描述系统的主要功能模块及其相互关系。
- 子层数据流图
对顶层数据流图中的某一功能模块进行分解，描述该模块的子功能及其相互关系。

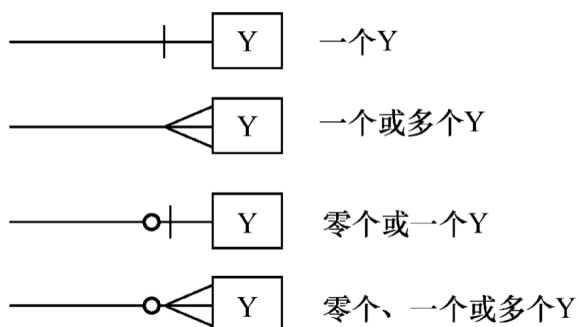
• 招生系统的分层数据流程图



3.2.3. 数据建模

在结构化分析方法中，使用实体—关系建模技术来建立数据模型。

ER 图中仅包含 3 种相互关联的元素：数据对象（实体）、描述数据对象的属性及数据对象彼此间相互连接的关系。



3.2.4. 行为建模

状态转换图（简称状态图）通过描绘系统的状态及引起系统状态转换的事件，来表示系统的行为。

状态可能有：初态（初始状态）、终态（最终状态）和中间态。

在一张状态图中只能有一个初态，而终态则可以有多，也可以没有。



事件：在某个特定时刻发生的事情，它是对引起系统做动作或从一个状态转换到另一个状态的外部事件的抽象。事件表达式的语法如下：
事件说明 [守卫条件] / 动作表达式

1. 事件说明：事件名（参数表）
2. 守卫条件是一个布尔表达式。
 - 有事件说明 & 守卫条件，两者都为真时才转状态
 - 只有守卫条件，守卫条件为真转化
3. 状态转换执行动作表达式

3.2.5. 数据字典

数据字典成为把 3 种分析模型黏合在一起的“黏合剂”，是分析模型的“核心”。

1. 词条描述

- 对于在数据流图中每一个被命名的图形元素均加以定义；
- 定义内容：图形元素的名字、别名或编号，图形元素类别（如加工、数据流、数据文件、数据元素、数据源点或数据汇点等）、描述、定义、位置等。

2. 数据结构描述

在数据字典的编制中，分析员最常用的描述数据结构的方式有定义式、Warnier 图等。

- 定义式。在数据流图中，数据流和数据文件都具有一定的数据结构，因此，必须以一种清晰、准确、无二义性的方式来描述数据结构。

数据结构定义式中的符号		
符 号	含 义	解 释
=	被定义为	例如， $x=a+b$ ，表示 x 由 a 和 b 组成
+	与	例如， $x=[a, b]$ ， $x=[a b]$ ，表示 x 由 a 或由 b 组成
[... , ...]	或	
[... ...]	或	
{ ... }	重复	例如， $x=\{a\}$ ，表示 x 由 0 个或多个 a 组成
$m\{...\}n$	重复	例如， $x=3\{a\}8$ ，表示 x 中至少出现 3 次 a ，至多出现 8 次 a
(...)	可选	例如， $x=(a)$ ，表示 a 可在 x 中出现，也可不出现
" a " "	基本数据元素	例如， $x="a"$ ，表示 x 为取值为 a 的数据元素
...	连接符	例如， $x=1..9$ ，表示 x 可取 1 到 9 之中的任一值

- Warnier 图。它用树形结构来描绘数据结构。

3.2.6. 加工规格说明

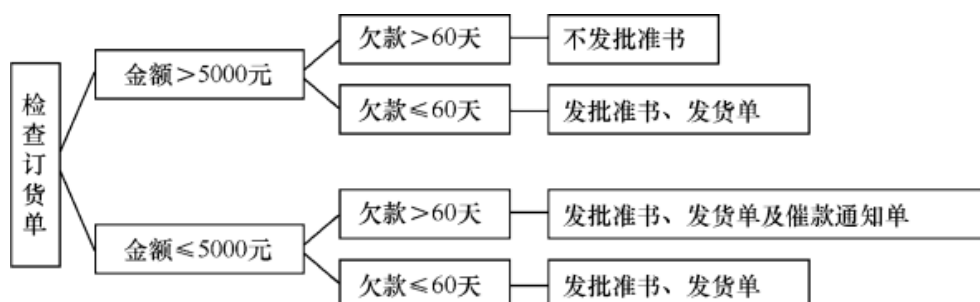
描述基本加工“做什么”

1. 决策表

如果表中有两条或更多的处理规则具有相同的动作，并且其条件项之间存在着某种关系，就可设法将它们合并。

改进的“检查订货单”的决策表					
		1	2		3
条件	订货单金额	>5000 元	—		≤5000 元
	偿还欠款情况	>60 天	≤60 天		>60 天
动作	在偿还欠款前不予批准	√			
	发出批准书		√		√
	发出发货单		√		√
	发催款通知书				√

2. 决策树



3.3. 小结

- 1.了解需求的重要性及需求分析阶段的目标及主要产物。
- 2.需求的类型：功能需求、非功能需求。
- 3.结构化分析的建模技术：ER图、数据流图、状态-迁移图、数据字典。

软件需求的主要类型及举例：

1. 功能需求：描述系统需要执行的功能。
2. 非功能需求：描述系统性能、安全性等非功能性的要求。
3. 业务需求：系统需要解决的业务问题。（支持在线支付业务）
4. 技术需求：与开发技术和工具相关的要求。（开发语言）

3.4. 系统需求规格说明

需求分析阶段的重要任务之一是根据分析的结果编写**需求规格说明**，经过**严格评审**并得到**用户确认**之后，作为这个阶段的**最终成果**。

Chapter 4: 结构化设计方法

4.1. 软件设计的概念及原则

1. 原则

①. 分而治之

- 模块并非分解得越小越好，否则成本降低，但是联结模块的成本会增加

②. 模块独立性

- 定义：是指软件系统中每个模块只涉及软件要求的具体的子功能（内聚性），而和软件系统中其它模块的接口是简单的（**耦合性**）。
- 有效的模块化使软件便于分工协作开发。
- 独立的模块比较容易测试和维护。
- 模块独立性比较强的模块应是**高内聚低耦合**的模块。

③. 提高抽象层次

- 设计开始时应尽量提高软件的抽象层次，按抽象级别从高到低进行软件设计。

④. 复用性设计

- 复用是指同一事物不做修改或稍加修改就可以多次重复使用。用于软件开发称为软件复用。

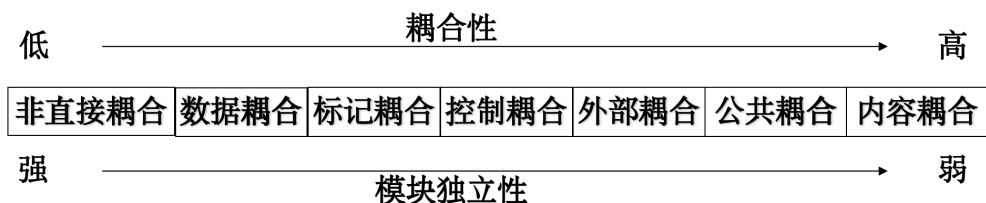
⑤. 灵活性设计

- 保证软件灵活性设计的关键是抽象。

在设计中引入灵活性的方法有：

- 降低耦合并提高内聚
- 建立抽象（创建有多态操作的接口和父类）
- 消除硬编码；
- 抛出异常
- 复用代码

4.1.1. 耦合性与模块独立性



1. 非直接耦合

- 模块独立性最强，通过主模块的控制和调用来实现

2. 数据耦合

- 一个模块访问另一个模块时,通过**简单数据参数**(不是控制参数、公共数据结构或外部变量)来交换输入、输出信息的。也是较理想的耦合。

3. 标记耦合

- 一组模块通过**参数表**传递记录信息,就是标记耦合。这个记录是某一数据结构的子结构,而不是简单变量。因此这些模块都必须清楚该记录的结构。

4. 控制耦合

- 一个模块通过传递**控制信息**来控制另一个模块的行为,这种耦合称为控制耦合。控制信息通常是某种标志或开关。

5. 外部耦合

- 两个模块通过**外部数据存储区**(文件、数据库或某种硬件设备)进行信息交换。

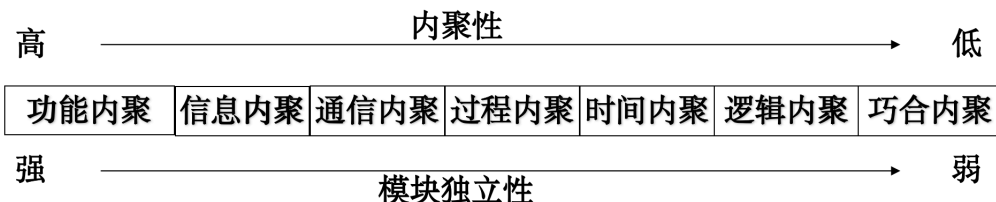
6. 公共耦合

- 两个模块通过访问同一个**全局数据区**(全局变量或公共数据结构)来交换信息。

7. 内容耦合

- 一个模块直接访问另一个模块的内部数据。
- 一个模块不通过正常入口转到另一模块内部。
- 两个模块有一部分程序代码重迭。(只可能出现在汇编语言中)。
- 一个模块有多个入口。

4.1.2. 内聚性与模块独立性



1. 巧合内聚

- 模块内各部分之间没有联系,或者即使有联系,这种联系也很松散。e.g. 类中多个操作函数间

2. 逻辑内聚

- if ... else ...

3. 时间内聚

- 时间内聚模块大多为多功能模块,但模块的各个功能的执行与时间有关,通常要求所有功能必须在同一时间段内执行。

4. 过程内聚

- 如果一个模块内的处理是相关的,而且必须以特定次序执行,则是过程内聚。e.g. 打开文件 → 读取文件

5. 通信内聚

- 如果一个模块内各功能部分使用了**相同的输入数据**或产生**相同的输出数据**，则称为通信内聚。

6. 信息内聚

- 如果一个模块各个功能都在**同一数据结构上操作**，每一项功能都有唯一的入口点，则称为信息内聚。

7. 功能内聚

- 该模块中所有部分都是为了完成一项具体功能而协同工作，紧密联系，不可分割的。

4.2. 结构化设计

4.2.1. 软件设计的任务

软件设计是将用户需求**准确地转化**成为最终的软件产品的**唯一途径**，在需求到构造之间起到了桥梁作用。

- 从工程管理的角度，可以将软件设计分为**概要设计阶段**和**详细设计阶段**。
- 从技术的角度，传统的结构化方法将软件设计划分为**体系结构设计**（软件的主要结构元素及其之间的关系）、**数据设计**（ER图）、**接口设计**（内、外部接口）和**过程设计**（算法及内部数据结构）4部分；

1. 体系结构设计：DFD

2. 数据设计：ER图、数据字典

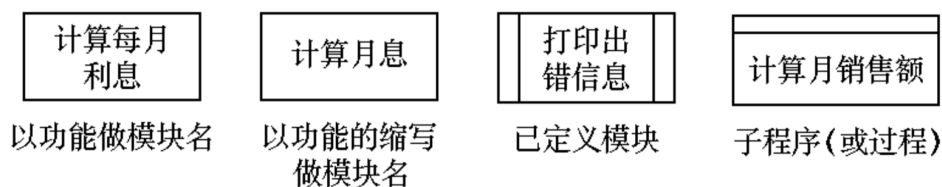
3. 接口设计：DFD

4. 过程设计：加工规格说明、控制规格说明（决策表、树）

4.2.2. 模块结构及其表示

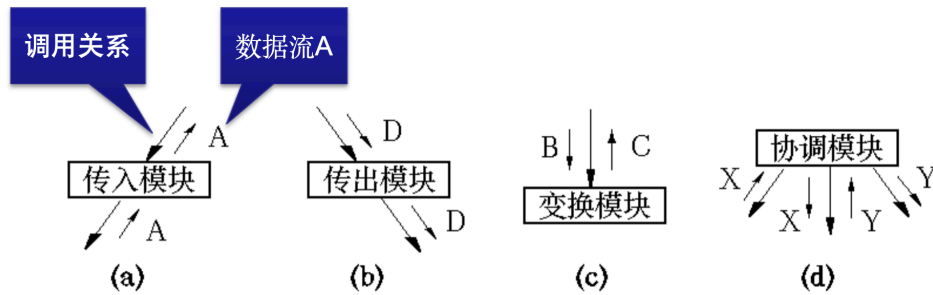
1. 模块

结构化程序设计中的函数和子程序都可称为模块。



2. 模块分类

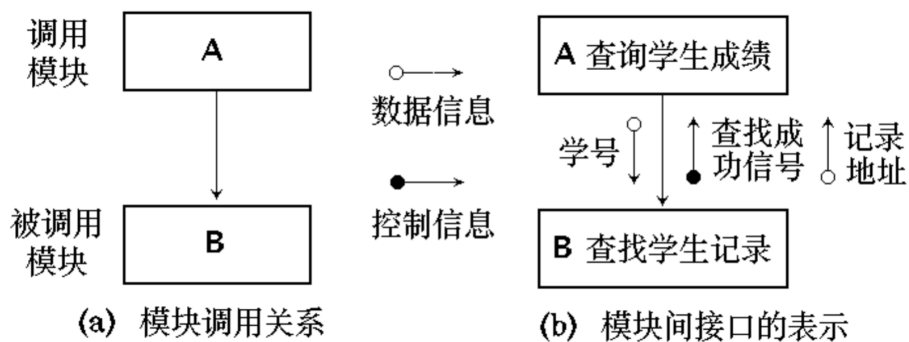
• 模块的分类



3. 模块结构（树状、网状）

4. 结构图

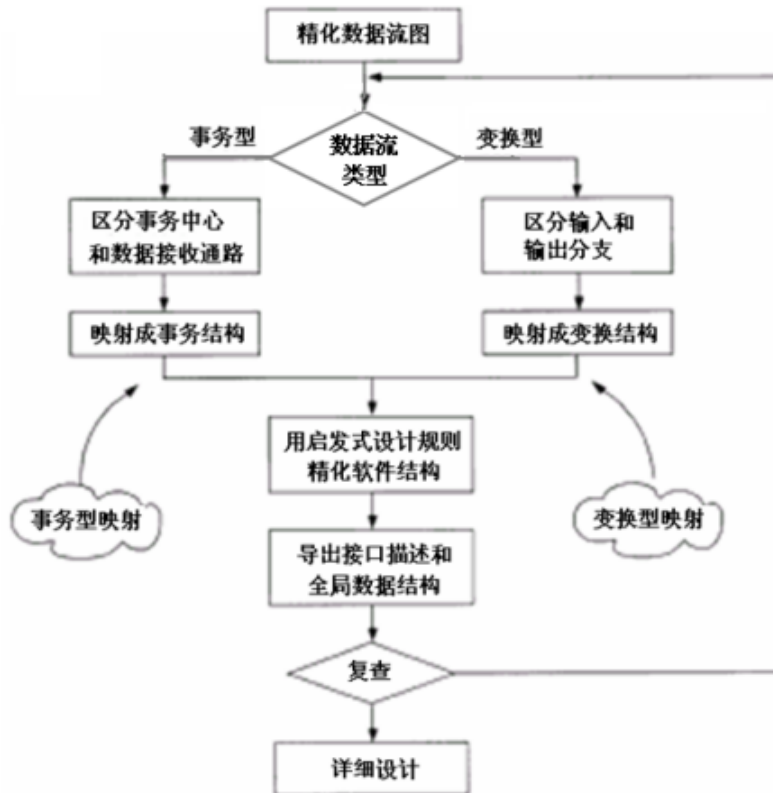
- 精确表达模块结构的图形表示工具。
 - ①. 模块的调用关系和接口（单箭头）
 - ②. 模块间的信息传递
 - ③. 条件调用和循环调用（条件：箭头根部加菱形；循环：箭头根部加弧形）
 - ④. 结构图的形态特征（上级调用下级，主从关系）



4.3. 体系结构设计

4.3.1. 基于数据流方法的设计过程

基于数据流的设计方法也称为**过程驱动**的设计方法

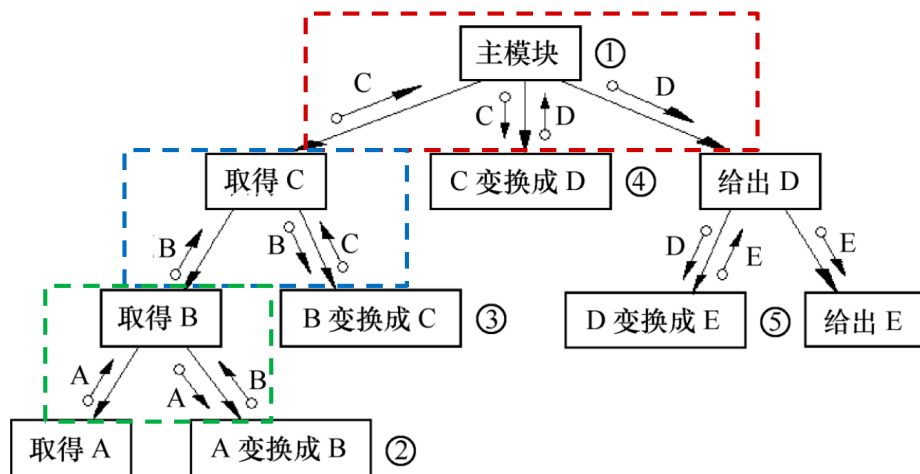


4.3.2. 典型的数据流类型和系统结构

数据流类型：变换型（all）、事务型（有明显事务特性）。

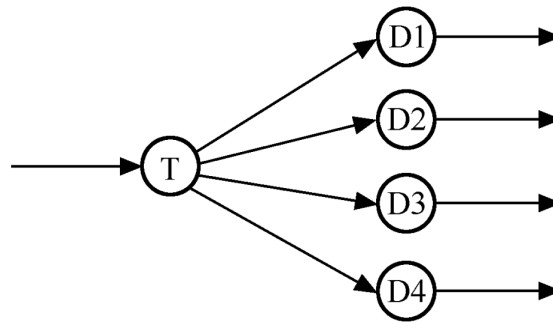
1. 变换型数据流

- 取得数据、变换数据和给出数据



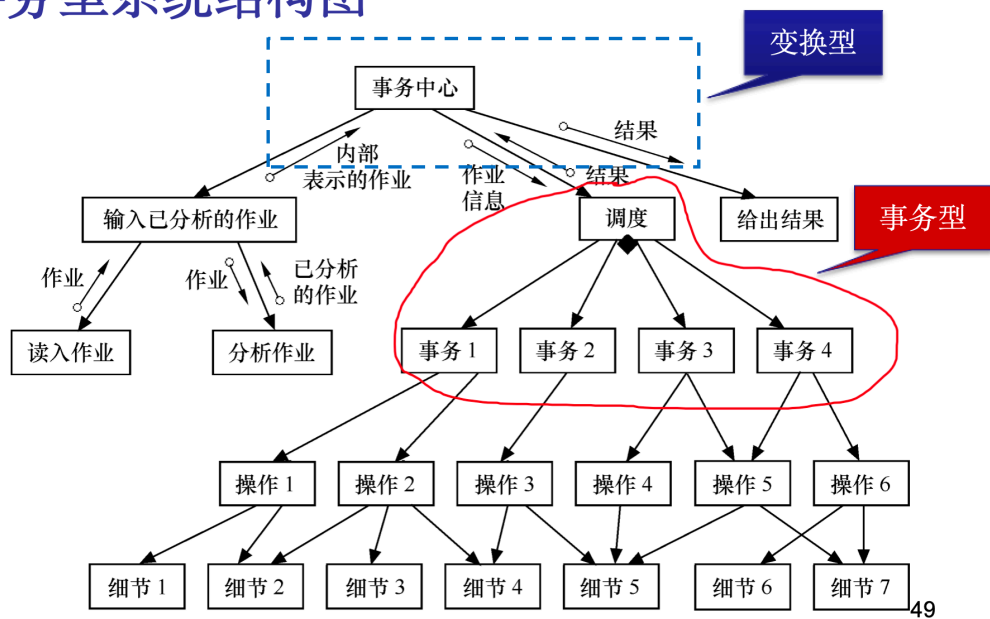
2. 事务型数据流

- 接受一项事务，根据事务处理的特点和性质，选择分派一个适当的处理单元，然后给出结果。
- 事务处理中心（分派部件）：选择分派任务的部分。



- 事务型系统结构图

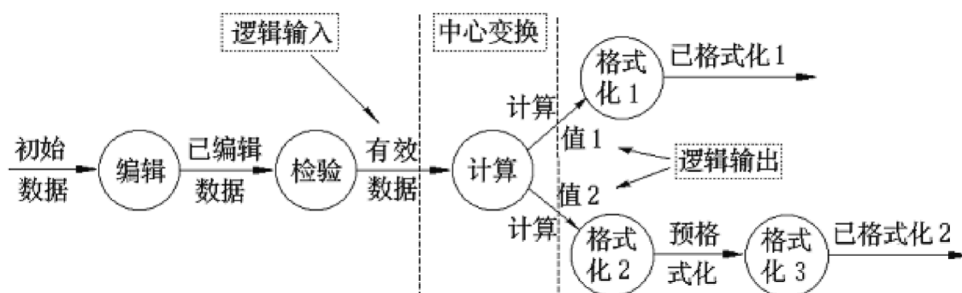
• 事务型系统结构图



4.3.3. 变换型映射方法

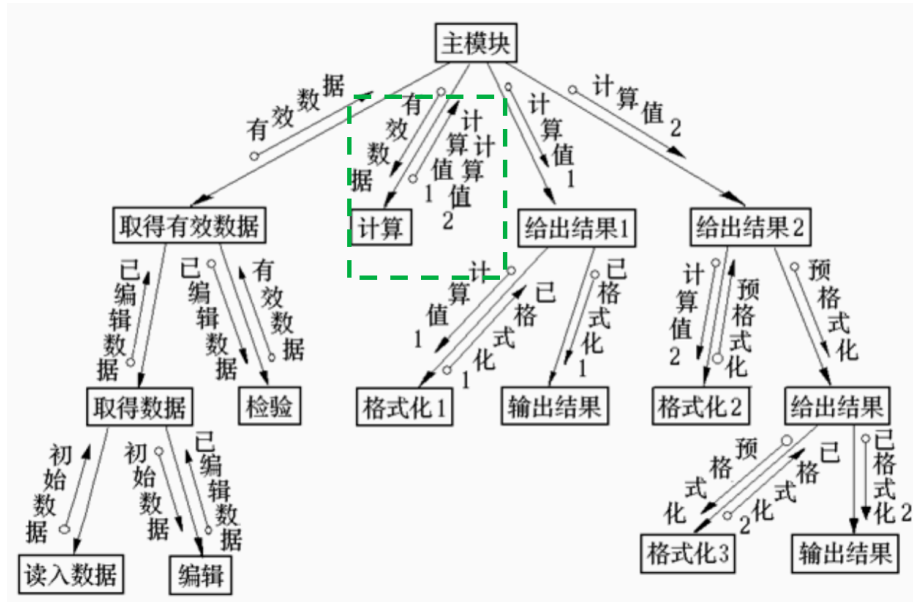
1. 变换分析方法的步骤

- ①. 重画数据流图。在需求分析阶段得到的数据流图侧重于描述系统如何加工数据，而重画数据流图的出发点是描述系统中的数据是如何流动的。
- ②. 在数据流图上区分系统的逻辑输入、逻辑输出和中心变换部分。



- ③. 进行一级分解，设计系统模块结构的顶层和第一层。

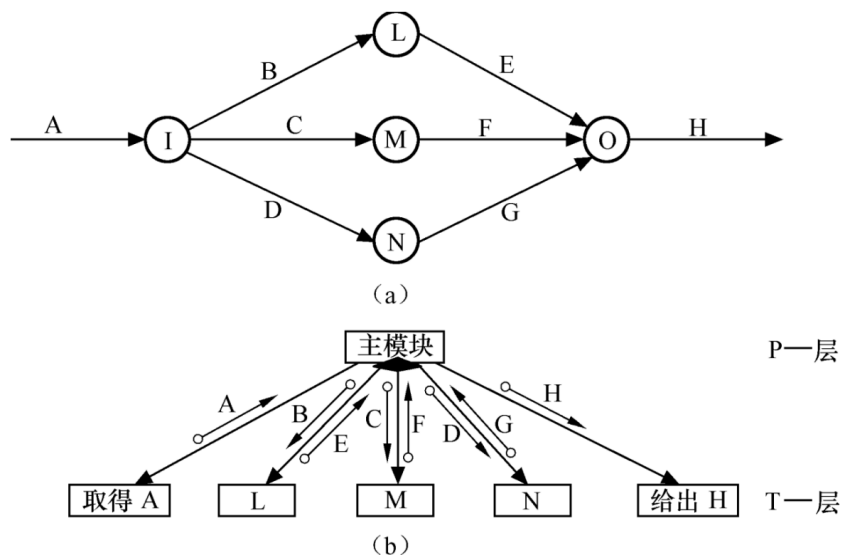
- a. 设计主模块，位于中心变换
- b. 第 1 层设计：
 - 逻辑输入——输入模块
 - 逻辑输出——输出模块
 - 中心变换——变换模块



- ④. 进行二级分解，设计中、下层模块
 - a. 为每一个输入模块、输出模块、变换模块设计它们的从属模块
 - b. 设计下层模块的顺序是任意的

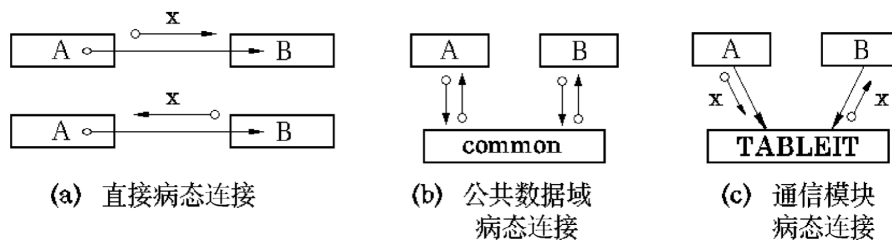
4.3.4. 事务型映射方法

每个处理单元为一个中间变换模块



4.3.5. 软件模块结构的改进方法

1. 模块功能的完善化
2. 消除重复功能，改善软件结构
 - 完全相似，结构完全一致，数据类型可能不同
 - 局部相似，不可合并
3. 模块的作用范围应在控制范围之内
 - 加黑框表示判定的作用范围
4. 尽可能减少高扇出（调用子模块数）结构（加中间层），随着深度增大扇入（上级模块数），（划分功能）
5. 避免或减少使用病态连接



6. 模块的大小要适中（代码在一页纸内）

首先找输入、输出边界，划分为输入数据、调度、输出数据，再细分三者。再根据模块调度边界移动模块，中间可以合并等。

4.4. 接口设计

1. 接口设计的依据是数据流图中的自动化系统边界

2. 接口设计主要包括 3 个方面：

- ①. 模块或软件构件间的接口设计；
- ②. 软件与其他软硬件系统之间的接口设计；
- ③. 软件与人（用户）之间的交互设计。

3. 人机交互（用户）界面是人机交互的主要方式

- 可使用性：简单、界面一致
- 灵活性：接口满足不同用户需求
- 可靠性：无故障使用时间

4. 人机交互的设计有若干准则：

一致性、操作步骤少、不要“哑播放”、提供 Undo 功能、减少人脑的记忆负担、提高学习效率。

4.5. 数据设计

4.5.1. 文件设计

1. 适合文件存储

- ①. 数据量较大的非结构化数据，如多媒体信息。
- ②. 数据量大，信息松散，如历史记录、档案文件等。
- ③. 非关系层次化数据，如系统配置文件。
- ④. 对数据的存取速度要求极高的情况。
- ⑤. 临时存放的数据。

2. 文件组织方式：顺序文件、直接存取、索引顺序文件、分区文件（放程序）、虚拟存储文件

4.5.2. 数据库设计

关系数据库最成熟，应用也最广泛。

4.6. 过程设计

- 概要设计的任务完成后，就进入详细设计阶段，也就是过程设计阶段。
- 此时，要决定各个模块的实现算法，并使用过程描述工具精确地描述这些算法。

表达过程规格说明的工具称为过程描述工具，可以将过程描述工具分为以下3类：

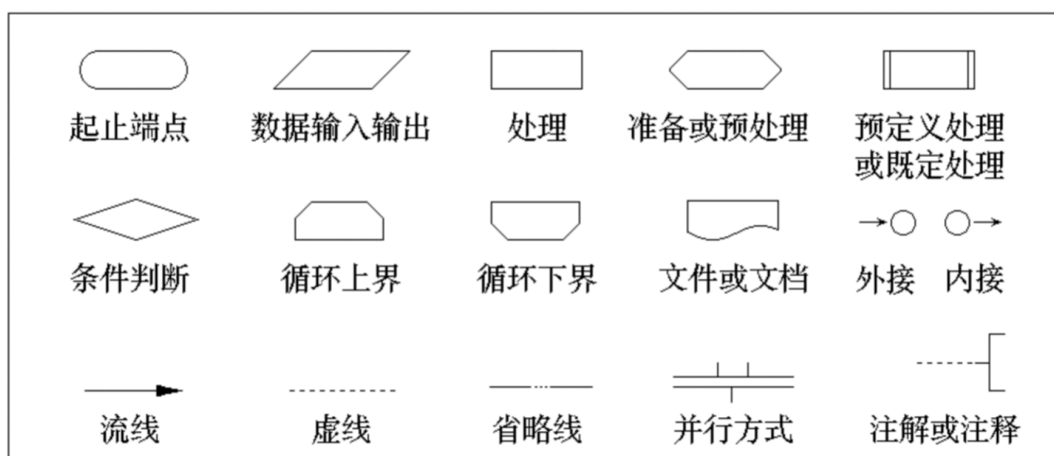
1. 图形工具（程序流程图、N-S图、PAD图、决策树）
2. 表格工具（决策表）
3. 语言工具（伪代码描述。类Pascal、类C语言描述算法）

4.6.1. 结构化程序设计

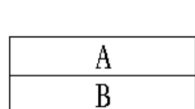
为了**克服软件危机**，引发了程序设计的一场革命，诞生了**结构化程序设计方法**

只用三种基本的控制结构“**顺序**”、“**选择**”和“**循环**”就能实现任何单入口和单出口的**没有“死循环”**的程序

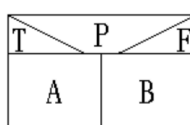
1. 程序流程图基本符号



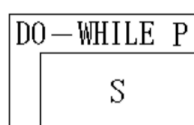
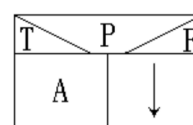
2. N-S 图（盒图）的基本控制结构



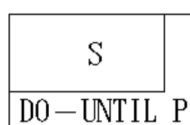
(a) 顺序型



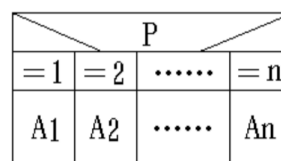
(b) 选择型



(c) WHILE 重复型



(d) UNTIL 重复型



(e) 多分支选择型
(CASE 型)

3. 伪代码的基本控制结构

- ①. 简单陈述句结构：避免复合语句。
- ②. 判定结构：IF_THEN_ELSE 或 CASE_OF 结构。
- ③. 重复结构：WHILE_DO 或 REPEAT_UNTIL 结构。

4.7. 小结

- 系统设计的两个阶段以及各自的主要输出物。
- 体系结构及其依赖性
- 好的设计的衡量：内聚和耦合。
- 软件设计原则。

Chapter 5: 面向对象方法与 UML

5.1. 面向对象的概念与开发方法

面向对象 = 对象 + 类 + 继承 + 消息通信

1. 对象

对象与类形式差别：对象名、类名

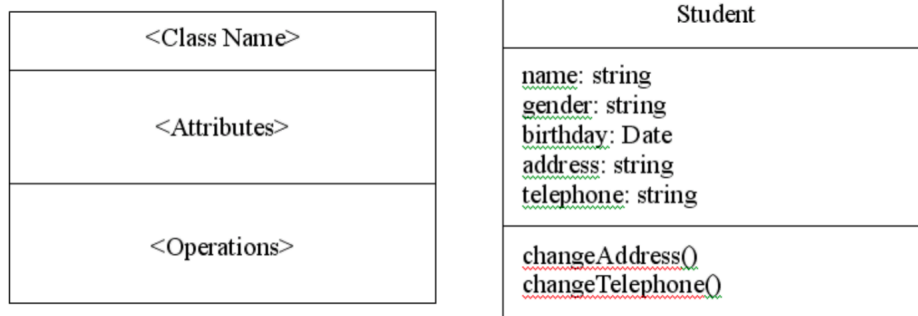
对象名有下列三种表示格式：

- ①. 对象名：类名
- ②. : 类名（用于尚未给对象命名的情况）
- ③. 对象名
- ④. 概念
 - a. 现实生活中对象指的是客观世界的实体
 - b. 程序中对象就是一组变量和相关方法的集合
- ⑤. 可以将程序中的**对象分为 5 类**：物理对象，角色，事件，交互，规格说明
 - 一个物理对象可以承担多个角色
 - 交互表示了在两个对象之间的关系

2. 类

类的图形

义为 Student。类的图形表示如图所示。



①. 封装

每个模块应该对外隐藏其内部实现，只暴露必要的接口。实现细节不能从类外访问。

②. 继承

通过继承复用已有代码，避免重复开发。

③. 多态

- 同一操作作用于不同对象产生不同结果
- 不同的模块可以通过相同接口调用不同实现，增强扩展性和灵活性。

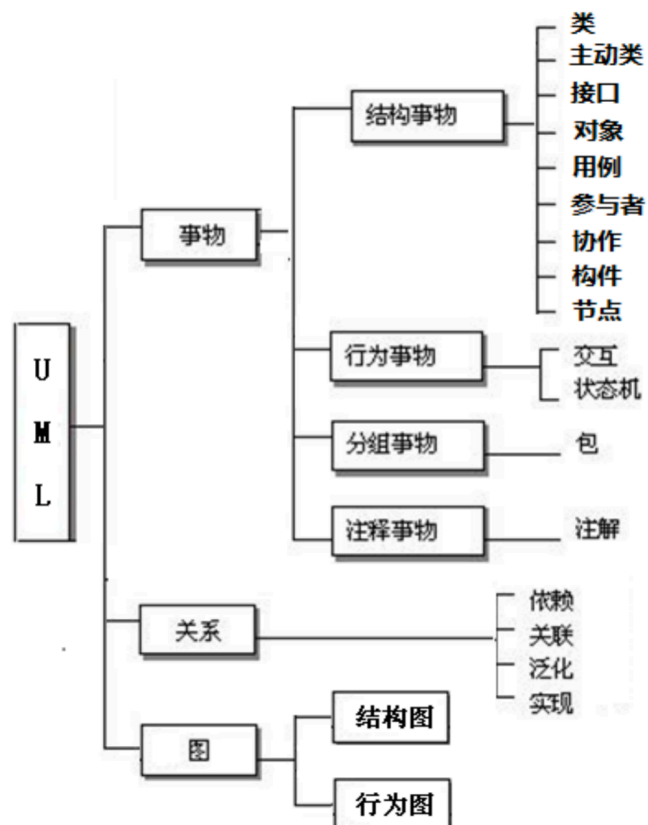
3. 消息通信

消息是一个对象与另一个对象的通信单元，是要求某个对象执行类中定义的某个操作的规格说明

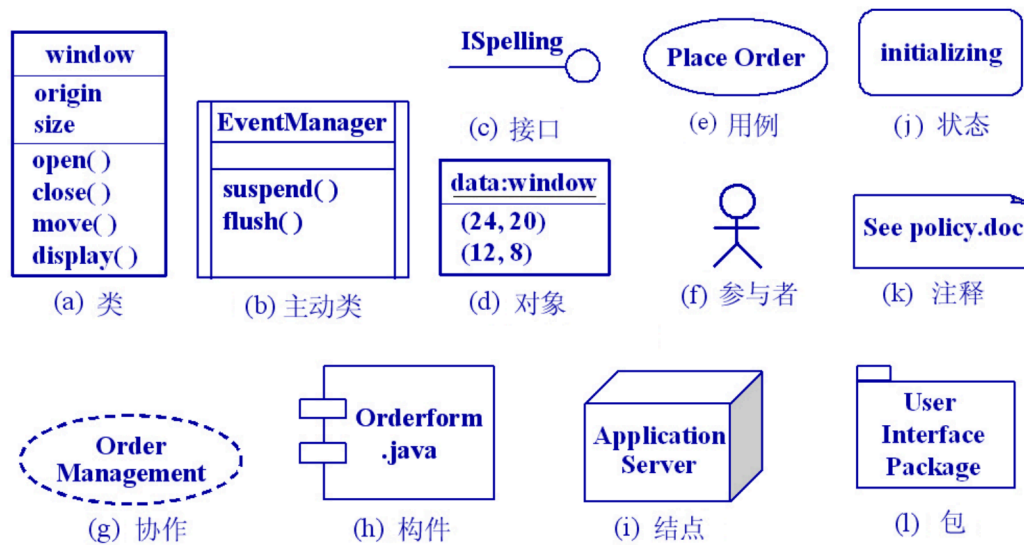
5.2. UML 简介

统一建模语言 UML 是一种广泛使用的面向对象的建模语言。

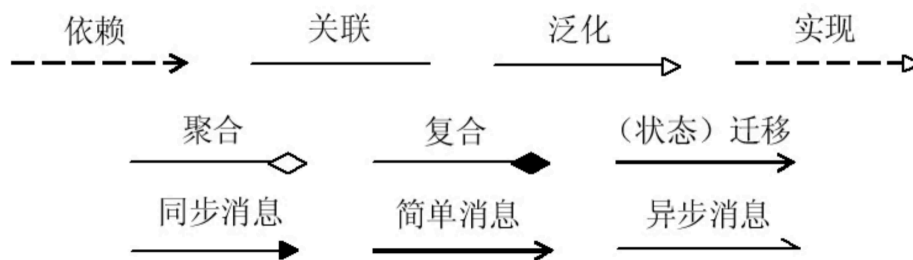
UML 模型由事物、关系和图组成。



5.3. UML 的事物



5.4. UML 的关系



5.4.1. 依赖关系

依赖是两个事物之间的语义关系，其中一个事物发生变化会影响到另一个事物的语义。从源指向目标事物，表示源依赖于目标事物。e.g. 调用、创建、实现、实例化...

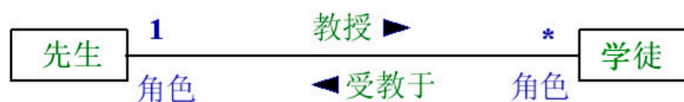
5.4.2. 关联关系

关联是一种结构关系，它描述了**两个或多个类的实例**之间的连接关系，是一种**特殊的依赖**。

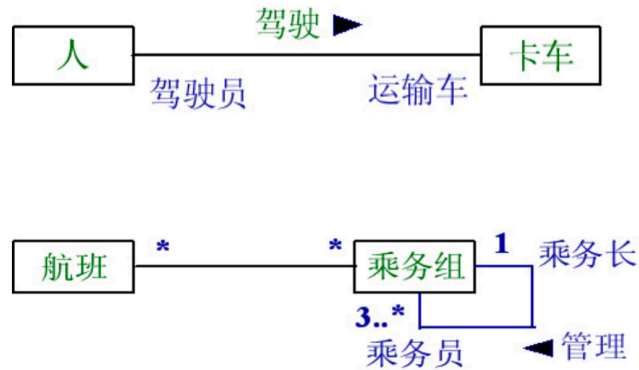
关联分为普通关联、限定关联、关联类，以及聚合与复合

1. 普通关联

- 最简单的关联关系，表示类与类之间的连接关系。分为二元关联和多元关联。



- 多重性：联系（0..1、1、0..*或*表示0到多个实例、1..*或1+表示1到多个实例）
- 角色：关联端点上还可以附加角色名。UML 还允许一个类与它自身关联。

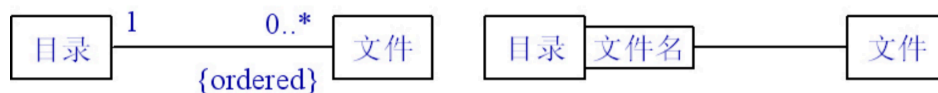


2. 多元关联

- 由一个菱形，以及由菱形引出的通向各个相关类的直线组成

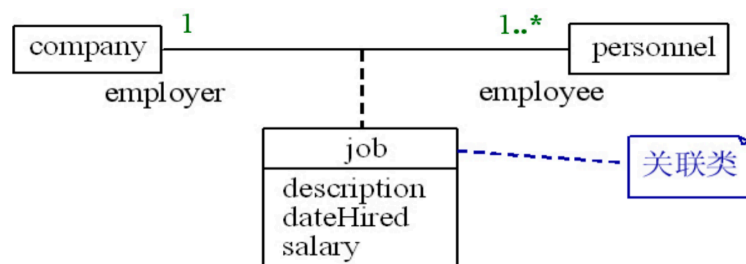
3. 限定关联

- 通常用在一对多或多对多的关联关系中，可以把模型中的多重性从一对多变成一对一，或将多对多简化成多对一。



4. 关联类

- 在某些情况下，需要对关联关系的语义做详细的定义、存储和访问，为此可以建立关联类，用来描述关联的属性。关联类通过一条虚线与关联连接



5. 聚合

也称为聚集，是一种特殊的关联。它描述了整体和部分之间的结构关系。

①. 共享聚合（整体类端+空心菱形）

在聚合关系中处于部分方的实例可同时参与多个处于整体方实例的构成

②. 复合聚合（整体类端+实心菱形）

部分类完全隶属于整体类，整体类与部分类级联删除。

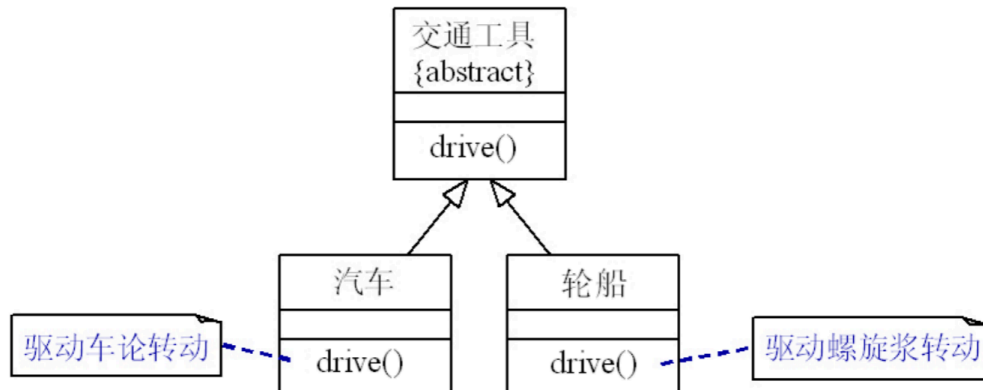
6. 导航

是关联关系的一种特性，它通过在关联的一个端点上加箭头来表示导航的方向。

5.4.3. 泛化关系

泛化关系就是一般类和特殊类之间的**继承关系**。空心箭头指向父类。

1. 普通泛化



- 普通泛化可以分为多重继承和单继承。多重继承是指一个子类可同时继承多个上层父类。

2. 受限泛化

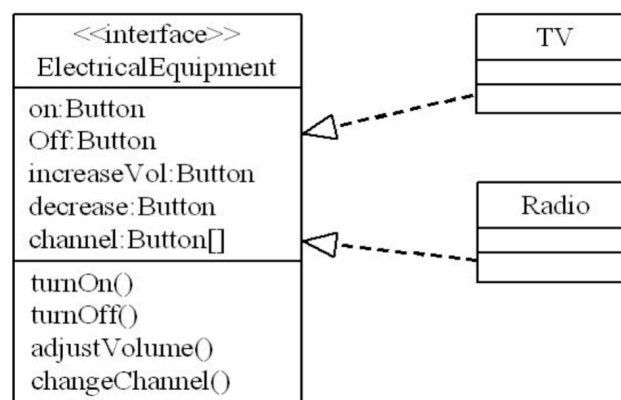
- 指泛化具有约束条件
- 一般有 4 种约束：交叠、不相交、完全和不完全

5.4.4. 实现关系

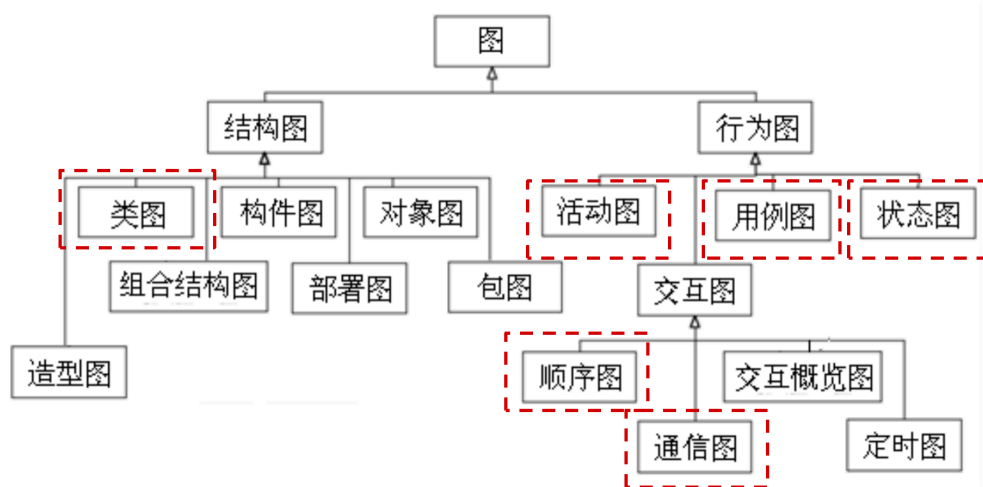
实现是泛化关系和依赖关系的结合，虚线空心箭头，指向接口、协作。

通常在以下两种情况出现实现关系：

1. 接口和实现它们的类或构件之间；
2. 用例和实现它们的协作之间



5.5. UML 的图



5.5.1. 用例图

1. 用例模型

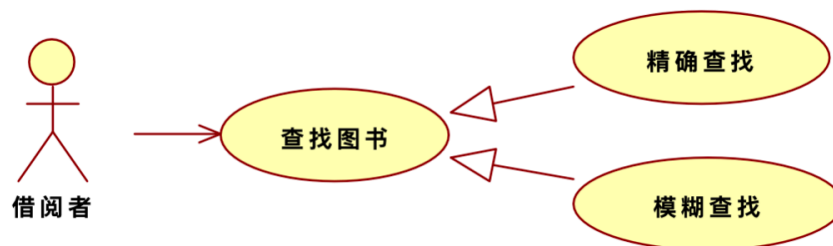
- 用例模型描述的是外部执行者所理解的系统功能。
- 一个用例模型由若干个用例图来描述，用例图的主要元素是用例和参与者。

2. 用例之间的关系

用例之间可以有**泛化**、**扩展**、**使用（包含）**三种关系

①. 泛化关系

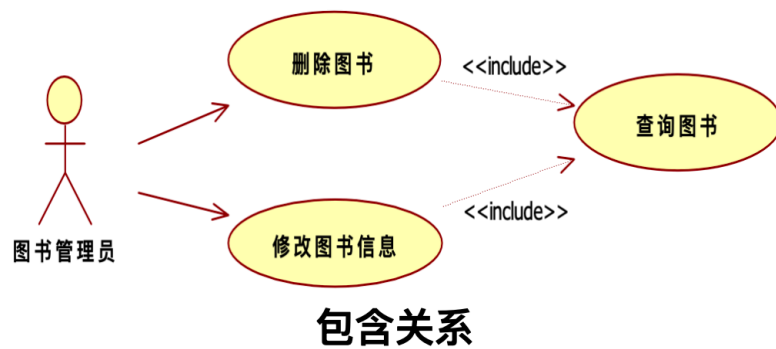
一个用例可以被特别**列举**为一个或多个子用例。



泛化关系

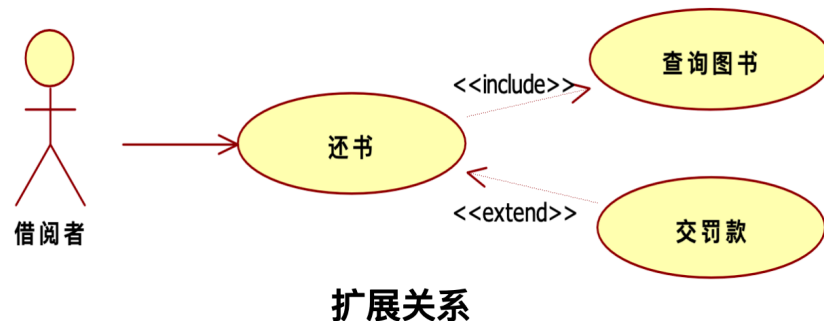
②. 使用（包含）关系

- 当一个用例使用另一个用例时，这两个用例之间就构成了使用关系。
- 当有一大块相似的动作存在于几个用例，又不想重复描述该动作，**将重复的部分分离为一个用例，两用例间关系称为使用关系。**



③. 扩展关系

向一个用例中加入一些新的动作后构成了另一个用例，**后者通过继承前者的一些行为得来**，通常把后者称为扩展用例。

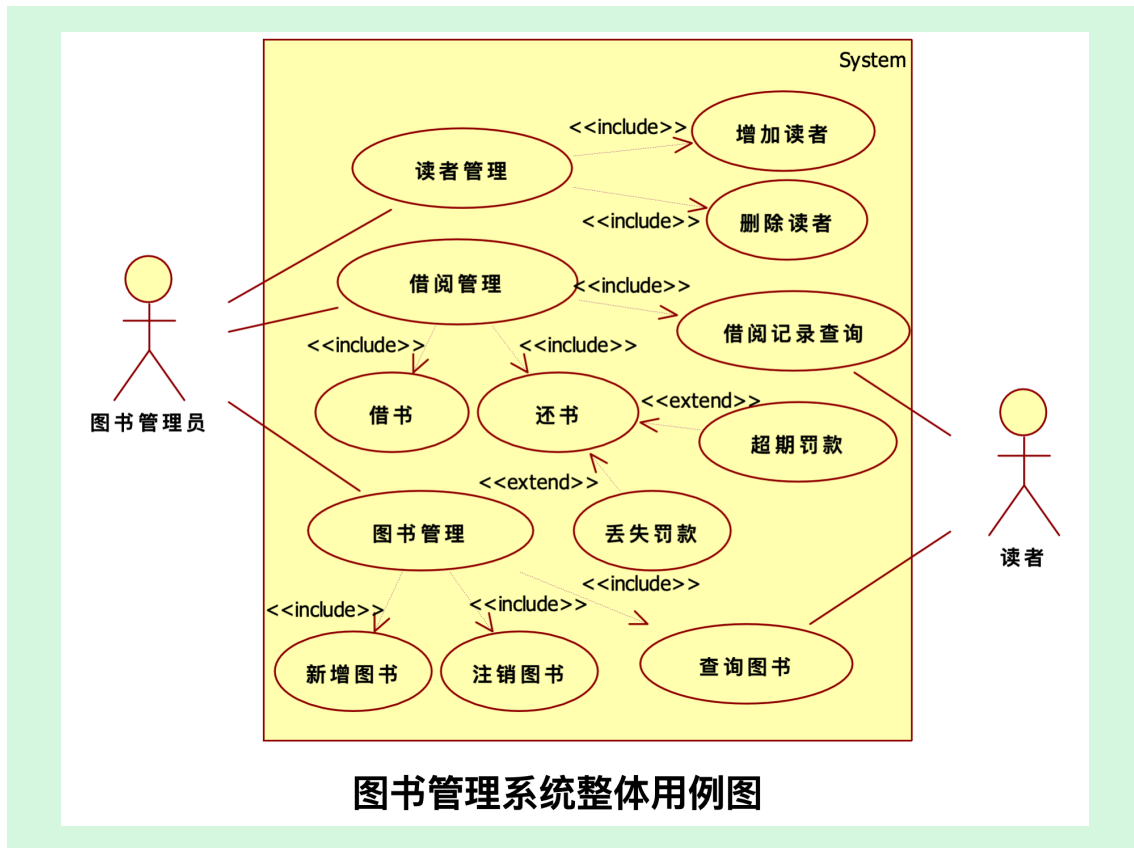


3. 绘制流程

- ① 找出系统外部参与者，确定系统边界和范围。
- ② 确定各参与者所期望的系统行为。
- ③ 把这些**系统行为命名为用例**。
- ④ 确定各用例之间的关系(泛化，包含，扩展)。
- ⑤ 绘制用例图。
- ⑥ 编制用例说明。
- ⑦ 对异常流程确定单独用例。

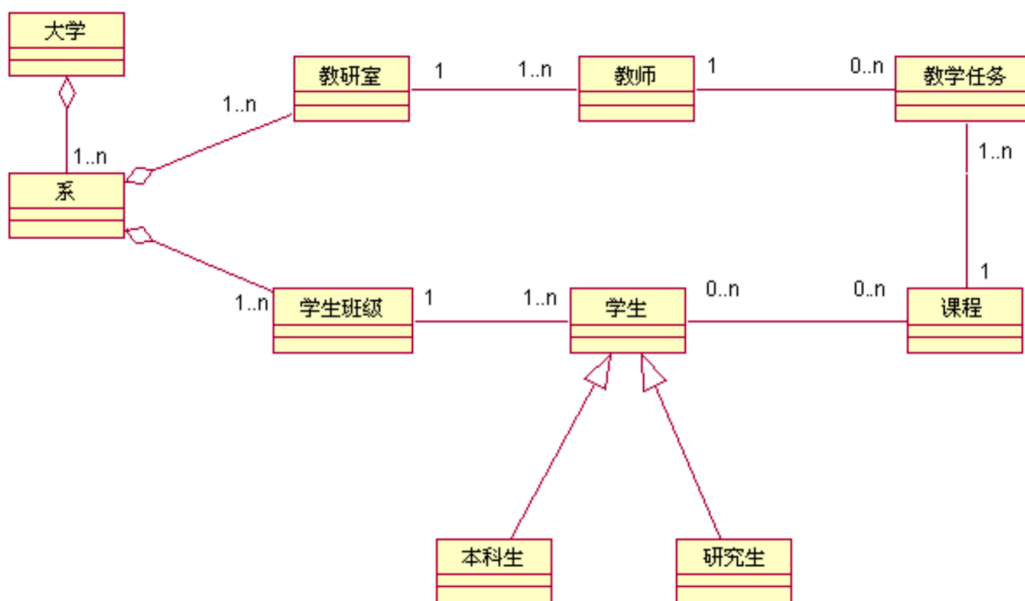
⑧ 优化用例图，解决用例之间的冲突和重复。

在图书管理系统中，管理员日常事务包括读者、图书、和借阅的管理，而读者可以查找图书，以及查询借阅记录。请画出图书管理系统的整体用例图。



5.5.2. 类图

类图描述类和类与类之间的静态关系，是静态模型，是构成其他图的基础。在类的建模中可以使用关联、聚合和泛化（继承）关系。



5.5.3. 顺序图

顺序图是强调**消息时间顺序**的交互图，它描述了对象之间传送消息的时间顺序，用于表示用例中的行为顺序。由**对象**、**生命线**、**消息**和**控制焦点（激活）**四个基本元素组成。

1. 组成元素

①. 对象- Object

- 参与者实例也是对象
- 顺序图中水平方向为对象维；
- 一般参与者和对象按从左到右顺序排列在顺序图的顶部；
- 对象的命名方式有三种：
 - a. 显示对象名和类名
 - b. 只显示类名（匿名对象）
 - c. 只显示对象名（不关心类）

②. 生命线- Lifeline

- 表示对象存在的时间。
- 如果对象生命期结束，则用注销符号表示

③. 消息- Message

- 对象间交互信息的方式
- 5 种消息：
 - a. 调用（同步消息）【发送者把消息发送后，等待，直到接收者返回控制，可以自调用】

$$\begin{array}{c} \text{return:=message(para)} \\ \hline \longrightarrow \end{array} \quad (5.1)$$

- a. 发送（异步消息）【消息发送后，发送者继续操作，不等待】

$$\begin{array}{c} \text{message(para)} \\ \hline \longrightarrow \end{array} \quad (5.2)$$

- a. 返回

$$\begin{array}{c} \text{message1(para)} \\ \hline \longleftarrow \end{array} \quad (5.3)$$

- a. 创建 【-->】
- b. 销毁 【用较大的叉号表示】

④. 控制焦点（激活）- Activation

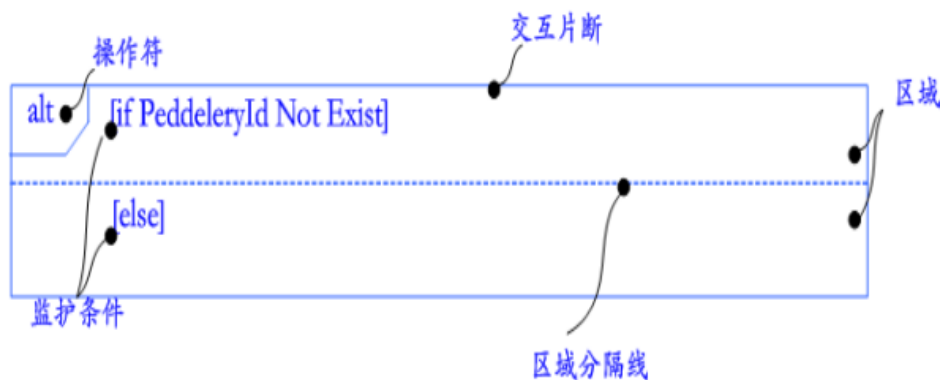
- 对象执行某个动作的时期

2. 消息格式

$$[\text{序号}][\text{条件}][\text{表达式}][\text{返回值 :=}]\text{消息名}([\text{params}]) \quad (5.4)$$

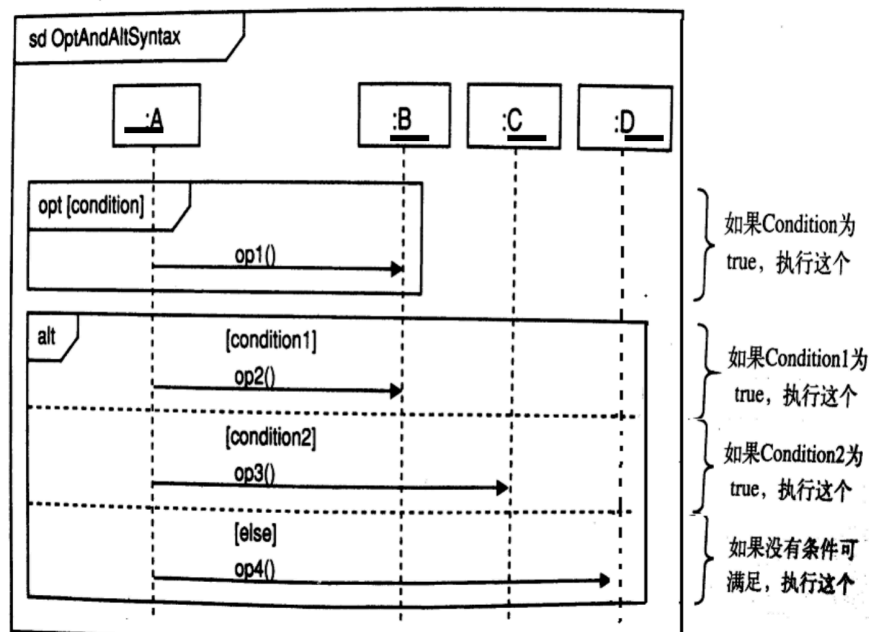
e.g.

- ①. 2: display(x,y) 简单消息
第 2 条消息，执行 display 函数
 - ②. 1.3.1:p:=find(specs) 嵌套消息
执行第 1 条消息第 3 步骤第 1 个子操作，调用 find，将结果赋值给 p
 - ③. 4 [x < 0] :invert(x,color) 条件消息
第 4 条消息，当 x < 0 时，调用 invert 函数
 - ④. 3.1 * :update() 循环消息
* 表示循环调用 update 函数
3. 顺序图中，对象的行为有循环和分支两种方式，为了表示这两种行为，引入了交互片段、区域和操作符的概念。



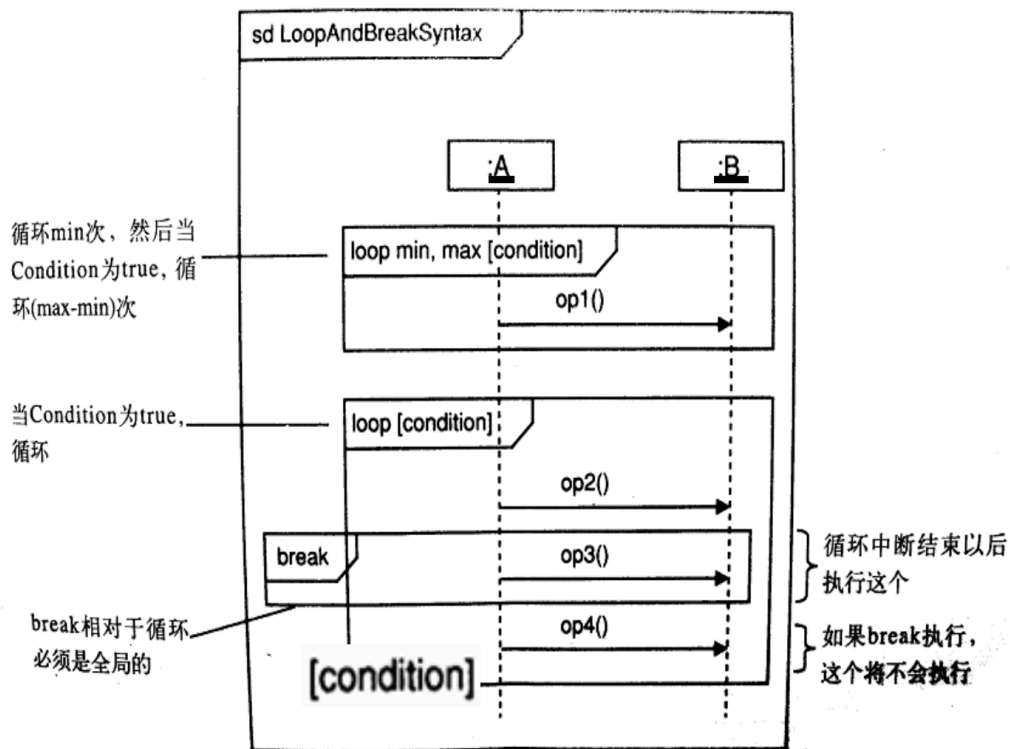
每一个交互片段都有一个操作符，决定其执行方式。

- ①. 表分支的操作符
 - a. alt: 支持多条件
 - b. opt: 支持单条件



- ②. 表循环的操作符 loop

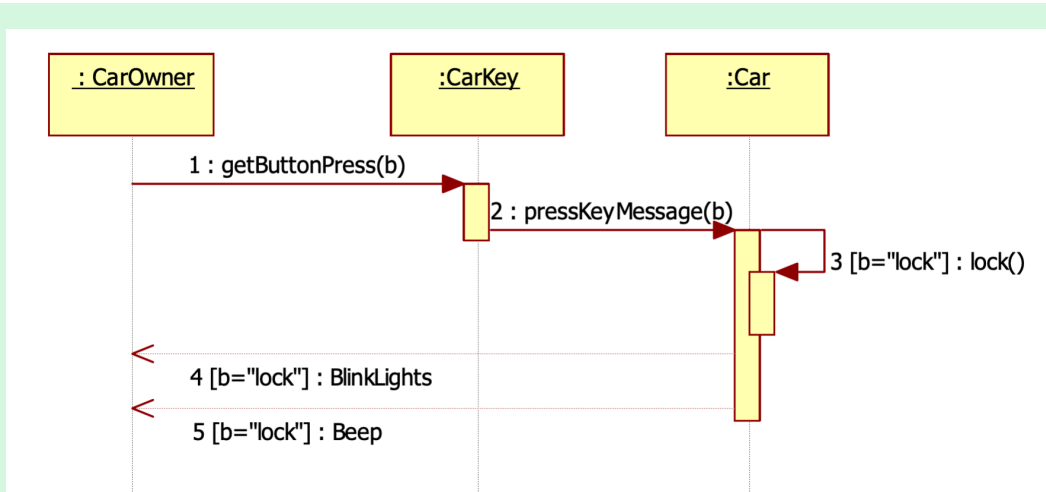
4. break



5. 顺序图构建

- ①. 为每一个用例编写事件流，包括一个基本事件流和若干可选事件流或异常流；
- ②. 识别事件流中参与交互的对象；
- ③. 为每个事件流绘制顺序图，包括为每个对象设置生命线，即确定哪些对象存在于整个交互过程中，哪些对象在交互过程中被创建和撤销；
- ④. 从引发这个交互过程的初始消息开始，在生命线之间自顶向下依次画出随后的各个消息；
- ⑤. 如果需要，可以画上交互片段、交互引用和约束等。

【例】大家应该熟悉能够遥控锁车、开锁、打开后备箱的钥匙。当按下“锁车”按钮的时候，汽车会自动上锁，闪动一下车灯并发出一声蜂鸣，说明车门上锁了。这里涉及到三个类：CarOwner（车主）、CarKey（车钥匙）和Car（车）。

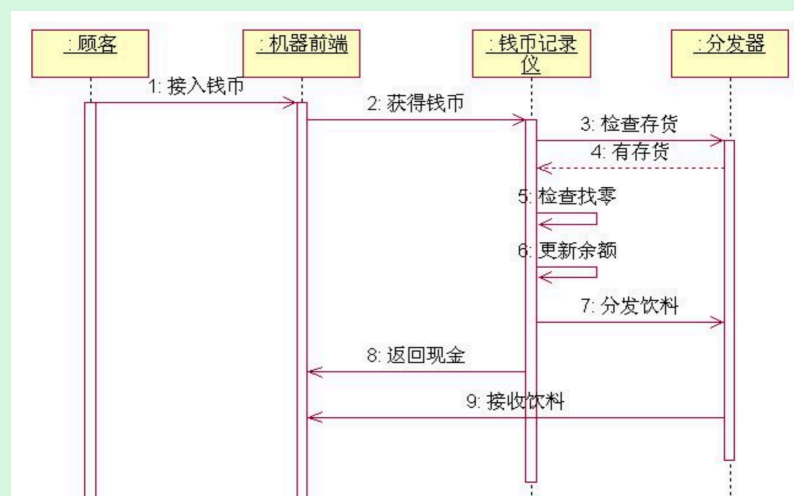


该建模场景的描述如下:

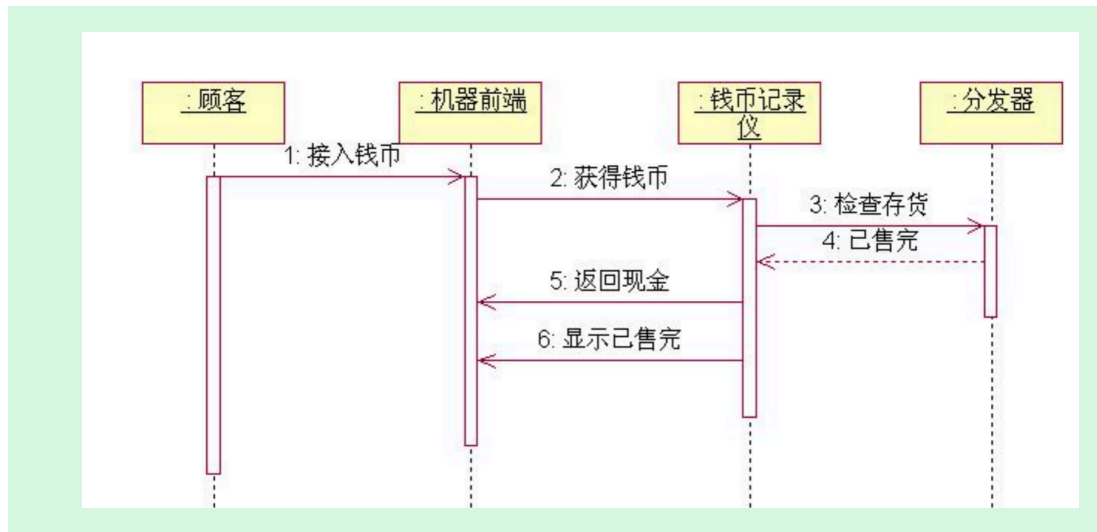
- ①. 从 CarOwner 到 CarKey 的请求, 要求 CarKey 实现 getButtonPress(b)操作, 登记下 CarOwner 按下的按钮 (通常用 b 引用)。
- ②. CarKey 发送消息给 Car, 通知 Car 实现其 pressKeyMessage(b)操作, 如果按下的按钮 b 是"lock", Car 就会向自己发送执行 lock()操作的请求。然后, Car 发送两个信号 BlinkLights 和 Beep 给 CarOwner。

以饮料自动销售系统为例, 对“买饮料”的 2 种场景 (买到饮料的场景和饮料“已售完”的场景) 进行建模, 对每一个场景, 我们绘制其对应的顺序图。下面是买到饮料的一般事件流:

- ①. 顾客从机器的前端钱币口投入钱币, 然后选择想要的饮料;
- ②. 钱币到达钱币记录仪, 记录仪更新自己的存储;
- ③. 记录仪通知分配器分发饮料到机器前端;
- ④. 买到饮料



- ②. 未买到



5.6. 状态图

状态图符号：

1. 起始点、终点：实心圆、牛眼圆
2. 状态：圆角矩形
3. 转换：箭头（事件表达式：触发事件[监护状态]/动作）
4. 分支：菱形（监护条件的取值产生分支）

绘制状态图的一般步骤是：

1. 寻找主要的状态；
2. 寻找外部事件，以便确定状态之间的转换；
3. 详细描述每个状态和转换；
4. 把简单状态图转换为复合状态图。

飞机票有以下 4 种状态：无预订、部分预订、预订完、预订关闭。

1. 在刚确定飞行计划时，显然没有任何预订，且在顾客预订机票之前都将处于“无预订”状态。
2. 对于订座而言，有“部分预订”和“预订完”两种状态。
3. 当航班快要起飞时，要“预订关闭”。

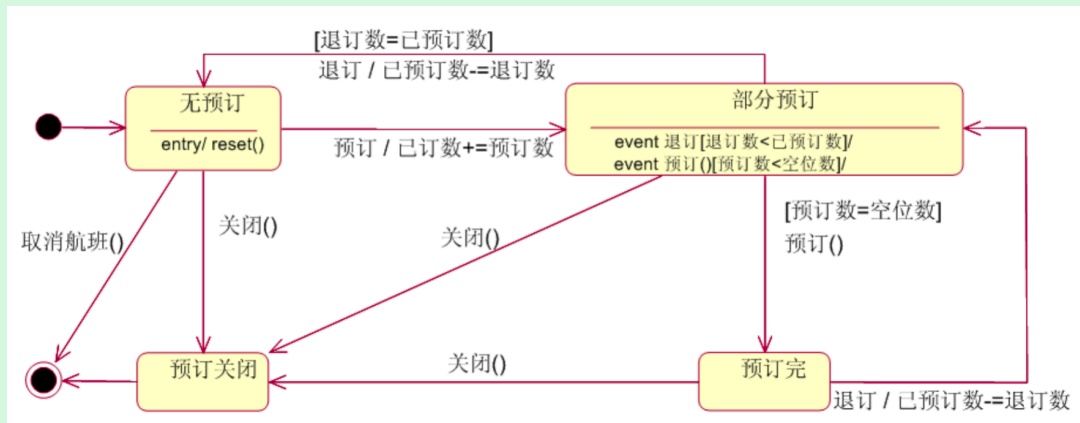
寻找外部事件：

1. 预订()：顾客预订机票。
2. 退订()：顾客退票。
3. 关闭()：机票管理员关闭订票系统。
4. 取消航班()：飞机调度人员取消飞行计划。

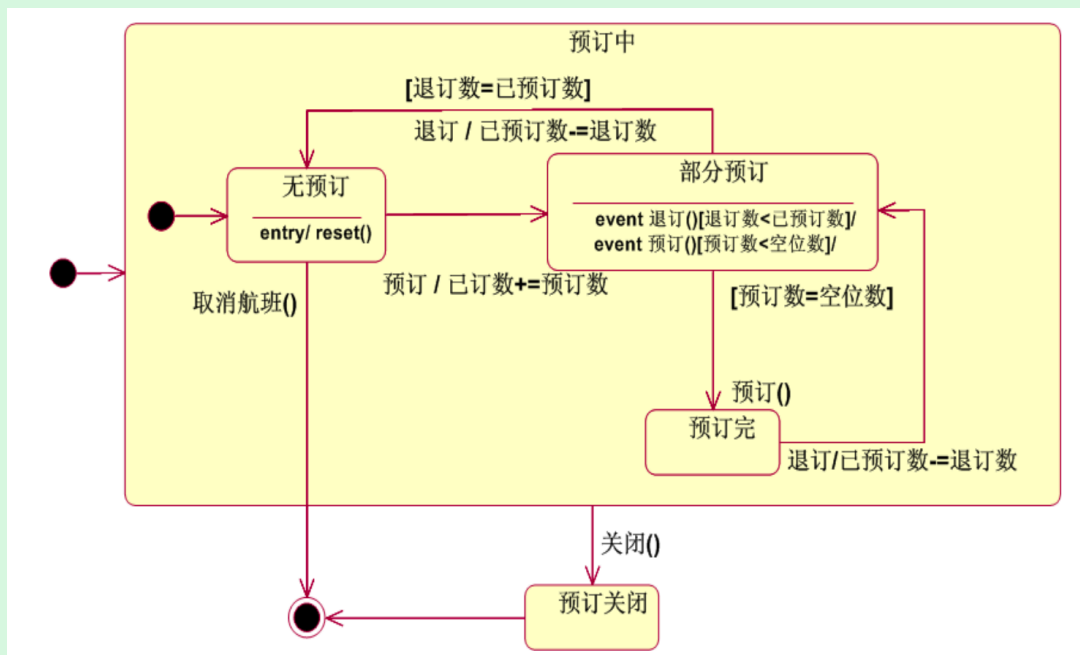
确定状态间的转换：

源\目标	无预订	部分预订	预订完	预订关闭
无预订		预订()	不直接转换	关闭()
部分预订	退订()事件发生后, 使预订人=0		预订(), 无空座	关闭()
预订完	不直接转换	退订()		关闭()
预订关闭	无转换	无转换	无转换	

详细描述后的状态图：



复合状态图：



5.6.1. 活动图

活动图是描述系统或业务的一系列活动构成的控制流，它描述了系统从一种活动转换到另一种活动的整个过程。

系统动态建模的两种主要形式；

1. 交互图强调对象与对象之间的交互信息；
2. 活动图强调从活动到活动的控制流程。

活动图的表示：

1. 初始节点和终点（与状态图表示符号一致）
 - 初始点为实心圆
 - 终点为牛眼圆，可能含多个

2. 活动节点

用来表示一个活动，一个活动表示多个动作的集合。

- 活动表示某流程中的任务的执行，也可以表示某算法过程中语句的执行。
- 动作是原子的，不能被分解。可视为活动的特例。时间不计。
- 活动节点用一个圆角矩形表示

活动节点的3种表示实例



3. 转换

当一个活动结束时，活动控制流就会传递给下一个活动节点，称为转换。用 $\rightarrow$ 表示。

4. 分支

活动图中，对于同一个触发事件，可以根据不同的监护条件转向不同的活动，每个可能的转移是一个分支。

5. 决策

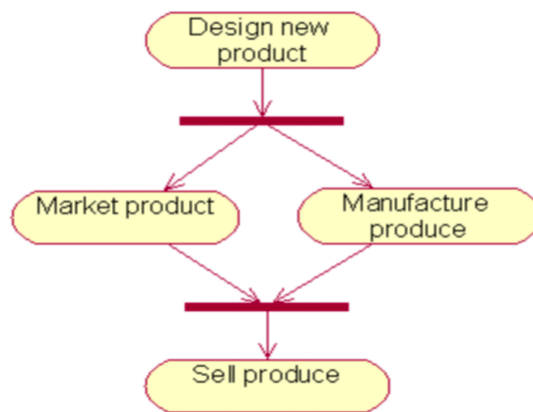
活动图中的控制节点。也称判决节点、判断节点。用菱形符号表示。

6. 合并

- 指两条或多条控制路径汇合的情况。用菱形符号表示。
- 具有两个或多个输入边和单一输出边。

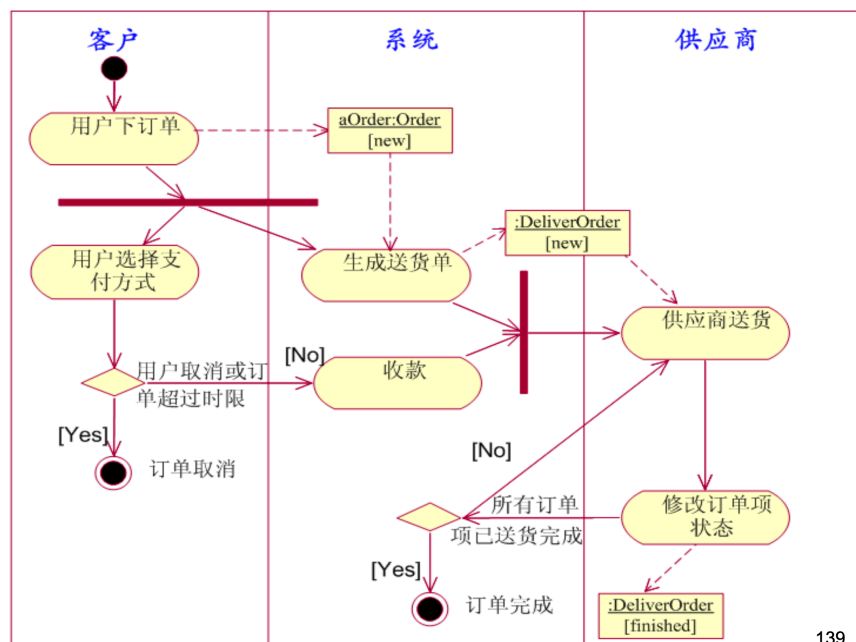
7. 分岔 (fork) 和汇合 (Join)

- 分岔表示一个控制流被两个或多个控制流代替，经过分岔后，这些控制流是**并发进行的**。(c.f 分支，分支是选择，分叉是并发)
- 汇合与分岔相反，表示两个或多个控制流被一个控制流代替。



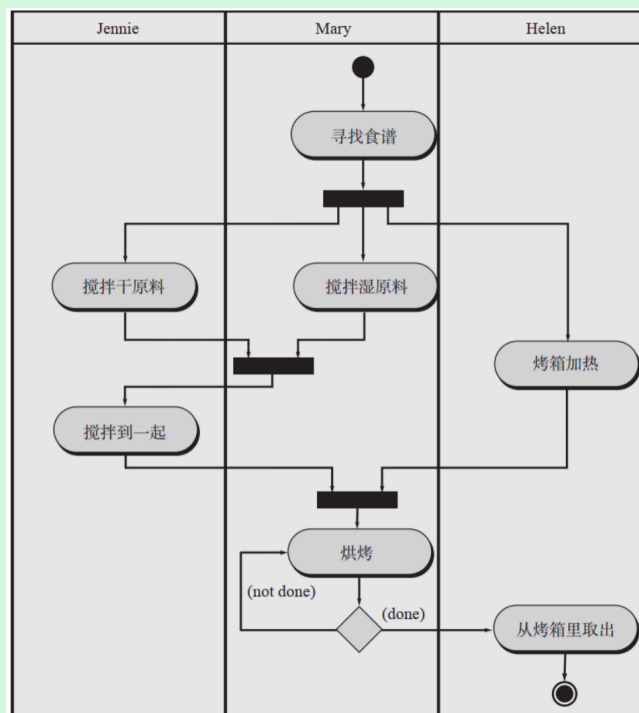
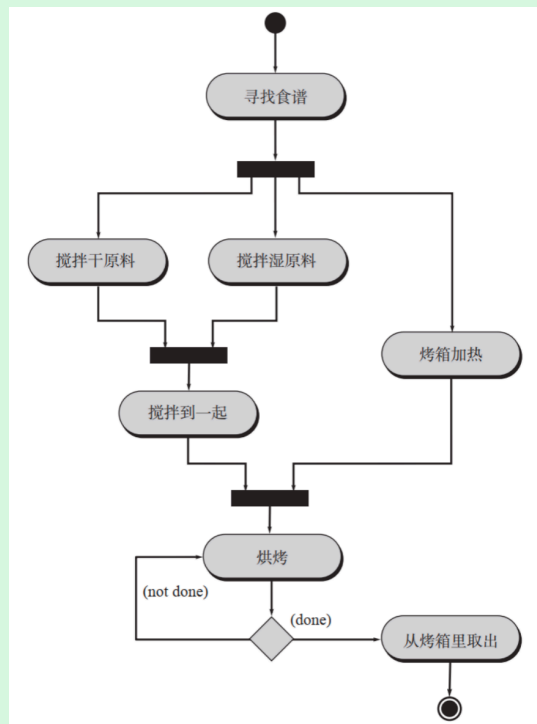
8. 泳道

泳道将活动图中的活动划分为若干组，并把每一组指定给负责这组活动的业务组织，即**对象**。每一组表示一个特定的类、人或部门，他们负责完成组内的活动。



139

烤蛋糕第一步是寻找食谱。一旦找到了食谱，就可以称量和搅拌干原料和湿原料，并且可以预热烤箱。干原料的搅拌可与湿原料的搅拌以及烤箱的预热并行进行。请画出烤蛋糕的活动图。



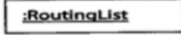
5.6.2. 构件图与部署图

- 构件图描述软件构件及构件之间的依赖关系，**显示代码的静态结构**。
- 构件是逻辑架构中定义的概念和功能(例如，类、对象及它们之间的关系)在物理架构中的实现。典型情况下，**构件是开发环境中的实现文件**。

软件构件可以是下述的任何一种构件。

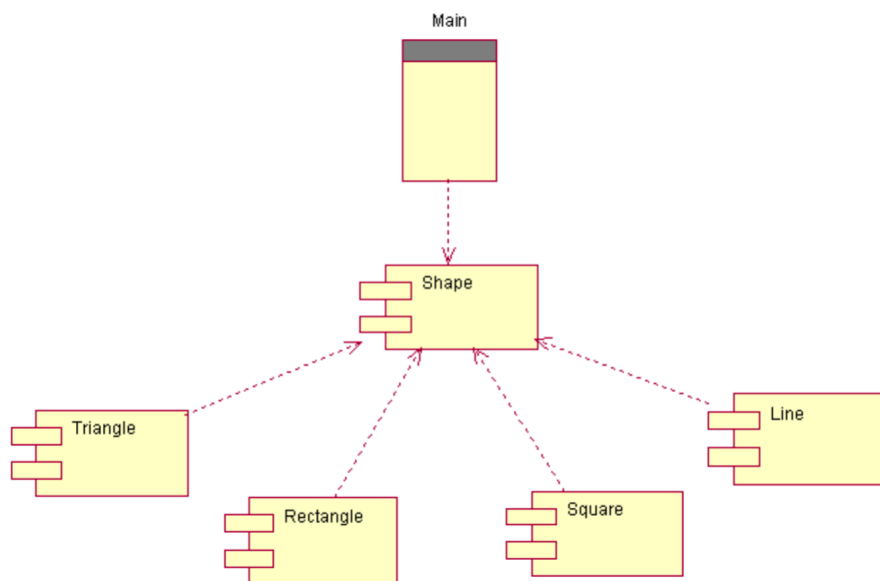
1. **源构件**：源构件仅在编译时才有意义。典型情况下，它是实现一个或多个类的源代码文件。
2. **二进制构件**：典型情况下，**二进制构件是对象代码**，它是源构件的编译结果。
3. **可执行构件**：可执行构件是一个可执行的程序文件，它是**链接所有二进制构件所得到的结果**。一个可执行构件代表在处理器(计算机)上运行的可执行单元。

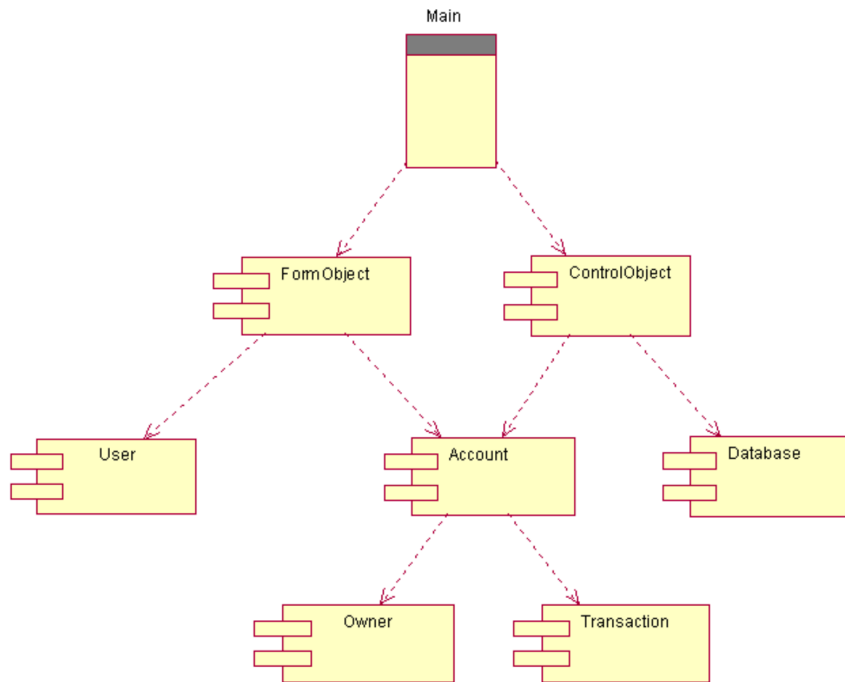
构件图的事物：

事物名称	含义	图例
构件	指系统中可替换的物理部分，构件名字(如图中的Dictionary)标在矩形中，提供了一组接口的实现。	 Dictionary
接口	外部可访问到的服务(如图中的Spell-check)。	 Spell-check
构件实例	节点实例上的构件的一个实例，冒号后是该构件实例的名字(如图中的RoutingList)。	 :RoutingList

构件图关系：

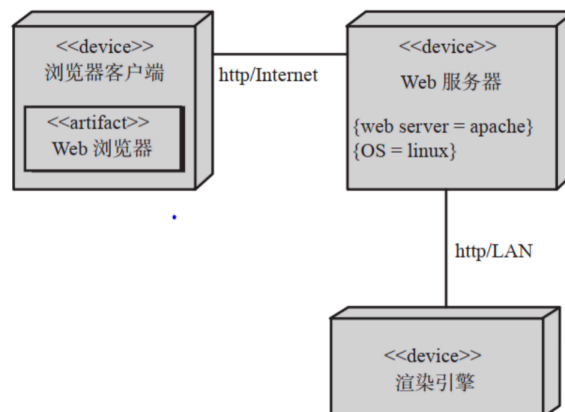
关系名称	含义	图例
实现关系	构件向外提供的服务。	
依赖关系	构件依赖外部提供的服务(由构件到接口)。	





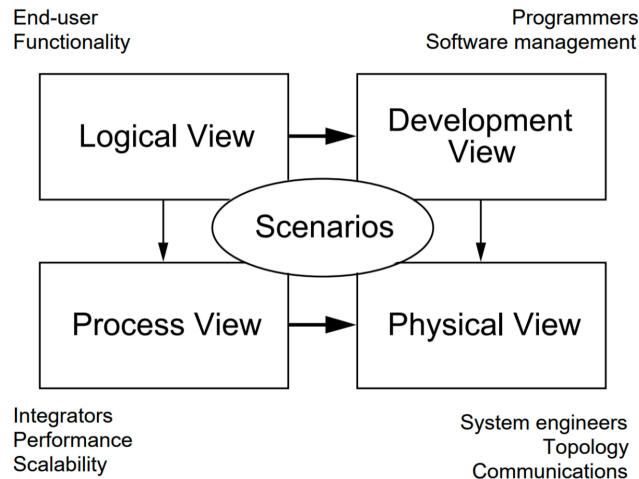
5.6.3. 部署图

- 部署图描述**处理器、设备和连接**，它显示系统硬件的物理拓扑结构及在此结构上执行的软件。
- 部署图可以显示**计算节点的拓扑结构和通信路径**、节点上运行的**软件**以及软件包含的**逻辑单元**。



5.7. UML 的 4+1 视图模型

系统架构视图模型：



English Term	中文翻译	核心关注点	主要 UML 图
Scenarios	场景视图	系统的关键用例和功能需求	用例图、活动图、顺序图
Logical View	逻辑视图	系统的功能抽象，面向对象的类、包、关系	类图、状态图、顺序图
Development View	开发视图	软件模块、组件、子系统的静态组织	组件图、包图
Process View	进程视图	运行时的进程、线程、通信、同步	活动图（并发）、顺序图
Physical View	物理视图	软件到硬件节点（服务器、网络）的映射	部署图

5.8. 小结

- (1) **UML的作用**：是为软件系统的制品进行描述 (**specifying**)、可视化 (**visualizing**)、构造 (**constructing**)、文档化 (**documenting**) 的一种语言。
- (2) **UML中的4 + 1视图**。
- (3) **UML中所包含的9种图形及各自的作用**。
- (4) 用例图的作用。
- (5) 用例图的主要构成部分。

1. 用例图

核心作用：描述外部参与者与系统功能的交互关系，明确系统边界和核心功能范围，是需求分析阶段的核心工具，用于对齐开发者与用户的需求共识。

2. 类图

核心作用：描述类的静态结构（属性、方法）及类间关系（关联、泛化、依赖、聚合 / 组合），是设计阶段的基础图形，为编码提供直接依据。

3. 顺序图

核心作用：按时间顺序描述对象间的消息交互流程，突出“时间维度”，清晰展示用例的具体执行步骤。

4. 通信图

核心作用：强调参与交互的对象组织关系（结构维度），与顺序图语义等价但视角不同，侧重对象间的连接而非时间顺序。

5. 状态图

核心作用：描述单个对象的生命周期，包括状态序列、状态转移的触发事件及伴随动作，聚焦对象的动态行为变化。

6. 活动图

核心作用：描述系统或业务的控制流（活动序列），支持分支、并发、泳道等特性，常用于业务流程或用例实现建模。

7. 构件图

核心作用：描述软件构件（源文件、二进制文件、可执行文件）及构件间的依赖关系，展示代码的静态物理结构。

8. 部署图

核心作用：描述硬件拓扑结构及软件在硬件上的部署关系，展示系统的物理运行环境。

9. 包图

核心作用：对 UML 模型元素（类、用例、构件等）进行分组组织，简化复杂模型的可读性，明确元素间的层次关系。

Chapter 6: 面向对象分析

6.1. 面向对象分析概述

6.1.1. 分析模型

- 分析建模使用文字和图表的综合形式描绘需求的数据、功能和行为。
- 软件工程师使用从客户那里提取的需求构建模型。
- 必须评审分析建模工作产品的正确性、完整性和一致性，必须反映所有共利益者的要求并建立一个可以从中导出设计的基础。
- 在技术层面上，软件工程开始于一系列的建模工作，**最终生成待开发软件的需求规格说明和全面的设计表示**。

6.1.2. 需求分析

1. 在整个需求建模过程中，软件工程师的主要关注点集中在“做什么”：
 - ①. 在特定环境下发生哪些用户交互？
 - ②. 系统处理什么对象？
 - ③. 系统必须执行什么功能？
 - ④. 系统显示什么行为？
 - ⑤. 定义什么接口？
 - ⑥. 有什么约束？
2. **需求模型必须实现 3 个主要目标：**
 - ①. 描述客户需要什么；
 - ②. 为软件设计奠定基础；
 - ③. 定义在软件完成后可以被确认的一组需求。
3. 分析模型在系统级描述和软件设计的差距之间建立桥梁。
4. **分析模型的所有元素都可以直接跟踪到设计模型。**
5. 分析的经验原则
 - ①. 模型应关注在问题域或业务域内可见的需求，抽象的级别应该相对高一些。
 - ②. 分析模型的每个元素都应能增加对软件需求的整体理解，并提供对信息域、功能和系统行为的深入理解。
 - ③. 关于基础结构和其他非功能的模型应推延到设计阶段再考虑。
 - ④. 最小化整个系统内的关联。
 - ⑤. 确认分析模型为所有共利益者都带来价值。
 - ⑥. 尽可能保持模型简洁。

6.2. 基于场景建模

使用 UML 分析建模，将从开发用例、活动图和泳道图形式的场景开始。

6.3. 基于类的建模

基于类建模表示了系统操作的对象、应用于对象间能有效控制的操作（也称为方法或服务）、这些对象间的关系以及已定义类之间的协作。

分析类的表现形式：

1. 外部实体，产生或使用基于计算机的系统的信息。
2. 事物（例如，报告、显示、字母、信号），问题信息域的一部分。
3. 偶发事件或事件，在系统操作环境内发生。
4. 角色，由和系统交互的人员扮演。
5. 组织单元（例如，部门、组、团队），和某个应用相关。
6. 场地，建立问题的环境和系统的整体功能。
7. 结构（例如，传感器、四轮交通工具、计算机），定义了对象的类或与对象相关的类。

6.4. 基于行为的建模

Chapter 7: 软件体系结构与设计模式

7.1. 软件体系结构的基本概念

什么是体系结构?

体系结构并非可运行的软件。确切地说,它是一种表达,使能够:

1. 在满足既定的需求方面下,分析设计有效性;
2. 在设计变更相对容易的阶段,考虑体系结构可能的选择方案;
3. 降低与软件构造相关的风险

体系结构为什么重要?

1. 软件体系结构的展示有助于对计算机系统开发感兴趣的所有利益相关者开展交流。
2. 体系结构突出了早期的设计决策,这些决策对随后所有的软件工程工作有深远的影响。并且对于系统最终的成功,与运行实体同等重要。
3. 体系结构“构建了一个相对小的、易于理解的模型,该模型描述了系统如何构成以及其构件如何一起工作”

Chapter 8: 面向对象设计

8.1. 面向对象设计过程与准则

面向对象设计准则 (6)

1. 模块化

- 面向对象方法中的模块是类、对象、接口、构件等。
- 数据和数据处理分离

2. 抽象

- 规格说明抽象 (类提供的外部接口)
- 参数化抽象 (模板将数据类型作为参数)

3. 信息隐藏

4. 弱耦合

- 对象是最基本的模块, 因此, 耦合主要指不同对象之间相互关联的紧密程度。

5. 强内聚

6. 可重用

面向对象设计原则 (4)

1. 开闭原则

- 模块应该对扩展具有开放性, 对修改具有封闭性。

2. 里氏替换原则

- 子类可以替换基类。将子类传递给构件来代替基类时, 使用基类的构件应该仍然能够正确完成其功能。(子类可覆盖父类抽象功能、扩展新功能, 重载的前置条件要更宽松)

3. 依赖倒置原则

- 通过抽象(接口或者抽象类)使各个类或模型的实现彼此独立, 不互相影响, 实现模块间的松耦合。

4. 接口分离原则

- 多个用户专用接口比一个通用接口要好。

8.2. 体系结构模块及依赖性

- 体系结构设计描述了建立计算机系统所需的数据结构和程序构件。一个好的体系结构设计要求软件模块的分层及编程标准的执行。
- 在面向对象软件中, 常见的软件模块有类、接口、包和构件。

8.2.1. 类及其依赖性

类和接口是程序的基本组成单元

继承依赖性

1. 一般来说，所有的继承都会引入**编译时依赖性**。依赖性是可传递的。
2. 运行时依赖性发生在继承类覆盖了父类方法的情况。

8.2.2. 接口及其依赖性

接口是不可直接实例化的特性集合的声明，即其对象不能直接实例化，需要通过类来实现，实现接口的类需要实现接口中声明的方法。

1. 将一个类和该类实现的接口之间的依赖性称为**实现依赖性**
2. 通过类（接口）和它所需接口之间的依赖关系来说明需求接口，这称为**使用依赖性**。

8.2.3. 包及其依赖性

包又可称为层或子系统，是表示组织类的一种方式，用于划分应用程序的逻辑模型。

包可以嵌套。外层包可以直接访问包括在它的嵌套包中的任何类。包还可以导入其他包。

两个包之间的依赖性来自于两个包中类之间的依赖性。

8.2.4. 构件及其依赖性

一个构件可以是一个编译的类，可以是一组编译的类，也可以是其他独立的部署单元，如一个文本文件、一个图片、一个数据文件、一个脚本等。

Chapter 9: 软件测试

评价一个程序和系统的特性或能力，并确定它是否达到预期的结果。软件测试就是以此为目的的任何行为。

测试的目的：(4)

1. 想以最少的时间和人力，系统地找出软件中潜在的各种错误和缺陷。如果我们成功地实施了测试，我们就能够发现软件中的错误。
2. 测试的附带收获是，它能够**证明软件的功能和性能与需求说明相符合**。
3. 实施测试收集到的测试结果数据为**可靠性分析**提供了依据。
4. 测试不能表明软件中不存在错误，它**只能说明软件中存在错误**。

软件测试应贯穿于软件定义与开发的整个期间，各阶段产生的文档都应该作为软件测试的对象。

1. 验证 vs 确认

- ①. 验证：是否满足规格说明
- ②. 确认：是否满足用户的要求

9.1. 单元测试

1. 单元测试是针对软件设计的最小单位——**程序模块**，进行正确性检验的测试工作
2. 可以把单元测试看成是从详细设计开始一直到系统构造完成贯穿于整个时期的一种活动
3. 只有通过了单元测试的模块，才可以把它们集成到一起进行集成测试。

单元测试的目的

1. 验证代码是与设计相符合的；
2. 跟踪需求和设计的实现；
3. 发现设计和需求中存在的缺陷；
4. 发现在编码过程中引入的错误。

单元测试的基本任务是保证模块中每条语句至少执行一次。

9.2. 集成

目标：根据实际情况对程序模块采用适当的集成测试策略组装起来，对**系统的接口以及集成后的功能进行正确性检验**的测试工作。

件集成测试依据概要设计说明和集成测试计划进行。

9.2.1. 冒烟测试

是对产品软件进行“每日构造”的一种常见方法。

步骤：

1. 将被翻译为代码的软件构件集成到构造中

2. 设计一系列测试来揭示错误，使得此构造不能正确地执行其功能
3. 将此构造与其他构造集成在一起，每天都对整个产品（以其当前的形式）进行冒烟测试

9.2.2. 回归测试

回归测试是重新运行已经进行过的测试子集，以确保变更没有引发非预期的副作用。

9.2.3. 面向对象的集成测试

对象的集成测试又称交互测试，目的是确保对象的消息传递能够正确进行。

9.2.4. 确认测试

1. Alpha 测试：由具有代表性的最终用户在开发者的场所进行。
2. Beta 测试：在一个或多个最终用户场所进行。

9.3. 系统测试

要证明被测系统的功能和结构的稳定性，还要有一些**非功能测试**，比如：性能测试、压力测试、可靠性测试等。

通过与系统的**需求规格说明**进行比较，检查软件是否存在与系统规格不符合或与之矛盾的地方，以验证软件系统的功能和性能等满足规格说明所指定的要求。

9.4. 调试

软件调试是在进行了成功的测试之后才开始的工作。它与软件测试不同，调试的任务是进一步诊断和改正程序中潜在的错误。

1. 确定程序中可疑错误的确切性质和位置。
2. 对程序（设计，编码）进行修改，排除这个错误。

Chapter 10: 软件测试用例设计

10.1. 黑盒测试（功能测试）

把测试对象看做一个黑盒，测试人员完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求和功能规格说明，检查程序的功能是否符合它的功能说明。

采用黑盒测试方法就意味着测试要在**软件的接口处**进行。

黑盒测试方法主要是为了发现以下几类错误

1. 是否有不正确或遗漏了的功能？
2. 在接口上，输入能否正确地接受？能否输出正确的结果？
3. 是否有数据结构错误或外部信息（例如数据文件）访问错误？
4. 性能上是否能够满足要求？
5. 是否有初始化或终止性错误？

理论上，用黑盒测试发现程序中的错误，必须在**所有可能的输入条件和输出条件**中确定测试数据，来检查程序是否都能产生正确的输出。

10.1.1. 黑盒测试用例设计方法

等价类划分是一种典型的黑盒测试方法。等价类划分方法把所有可能的输入数据，即程序的输入域划分成若干部分，然后从每一部分中选取少数代表性的数据做为测试用例。

1. 划分等价类
 - 有效等价类：对程序规格说明是合理的，有意义的输入数据构成的集合。
 - 无效等价类
2. 选取测试用例
 - ①. 一个等价类一个唯一编号；
 - ②. 设计一个新的测试用例，尽可能多地覆盖尚未被覆盖的有效等价类，重复直到所有的有效等价类都被覆盖；
 - ③. 设计一个新的测试用例，仅覆盖一个尚未被覆盖的无效等价类，重复直到所有的无效等价类都被覆盖。

10.2. 白盒测试

1. 把测试对象看做一个玻璃盒子，它允许测试人员利用程序内部的逻辑结构及有关信息，设计或选择测试用例，对程序所有逻辑路径进行测试。
2. 通过在不同点检查程序的状态，确定实际的状态是否与预期的状态一致。因此白盒测试又称为结构测试或逻辑驱动测试。

使用白盒测试方法，主要想对程序模块进行如下的检查：

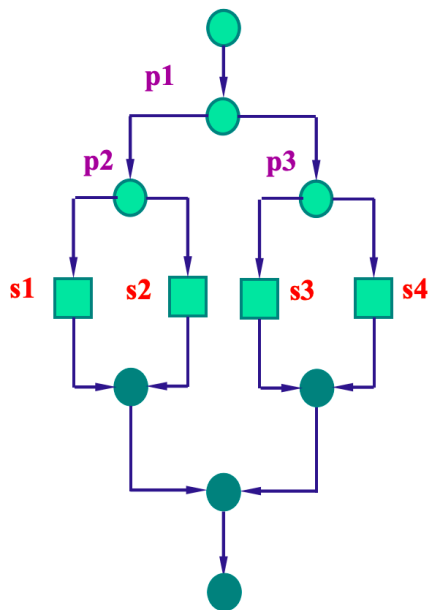
1. 对程序模块的所有独立的执行路径至少测试一次 — **路径覆盖测试**；
2. 对所有的逻辑判定，取“真”与取“假”的两种情况都至少测试一次 — **逻辑覆盖测试**；

3. 在循环的边界和运行界限内执行循环体 — **控制流测试**；
4. 测试内部数据结构的有效性 — 数据流测试、领域测试

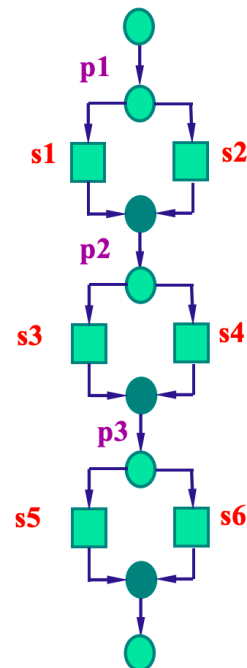
10.2.1. 分支结构

分支结构：

1. 嵌套型分支结构：若有 n 个判定语句，则存在 $n+1$ 条路径，需要 $n+1$ 个测试用例；
2. 连锁型分支结构：若有 n 个判定语句，需要有 2^n 个测试用例，覆盖它的 2^n 条路径。



嵌套型分支结构



连锁型分支结构

10.2.2. 循环结构

循环分为 4 种不同类型：简单循环、连锁循环、嵌套循环和非结构循环。

- 嵌套循环：对最内层循环做简单循环的全部测试。所有其他层的循环变量置为最小值；

10.2.3. 基本路径测试

基本路径测试方法把覆盖的路径数压缩到一定限度内，程序中的循环体最多只执行一次。

它是在**程序控制流图**的基础上，分析控制构造的**环路复杂性**，导出**基本可执行路径集合**，设计测试用例的方法。

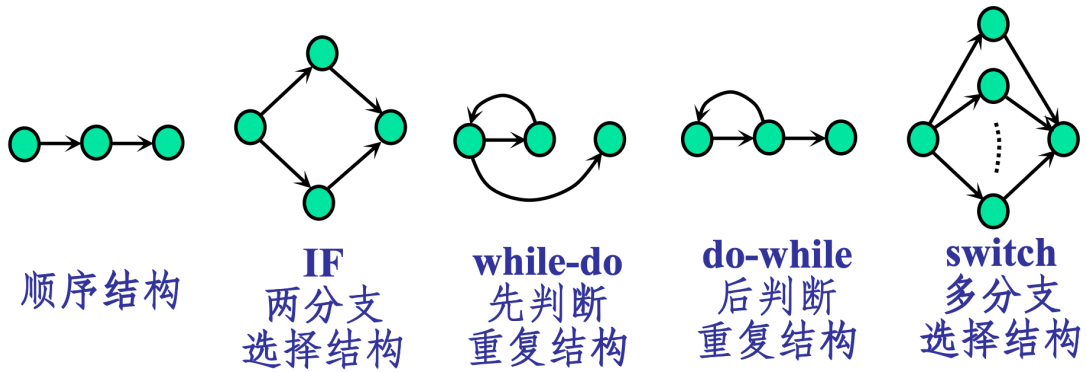
程序的控制流图：

1. 符号 $\circ$ 为控制流图的一个结点，表示一个或多个无分支源程序语句。箭头为边，表示控制流的方向。在选择或多分支结构中，分支的汇聚处应有一个汇聚结点。

2. 边和结点圈定的部分叫做区域，当对区域计数时，图形外的区域也应记为一个区域。
3. 程序环路复杂性：环复杂性=流图中区域个数

$$E - G + 2 \quad (\text{边数} - \text{点数} + 2)$$

$$P + 1 \quad (\text{单条件判断点} + 1) \quad (10.1)$$



- while-do，先判断后循环，所以先分支。

Chapter 11: 软件项目管理

软件项目管理涉及的几个主要方面是人员、产品、过程和项目，即所谓 4P

1. 人员管理

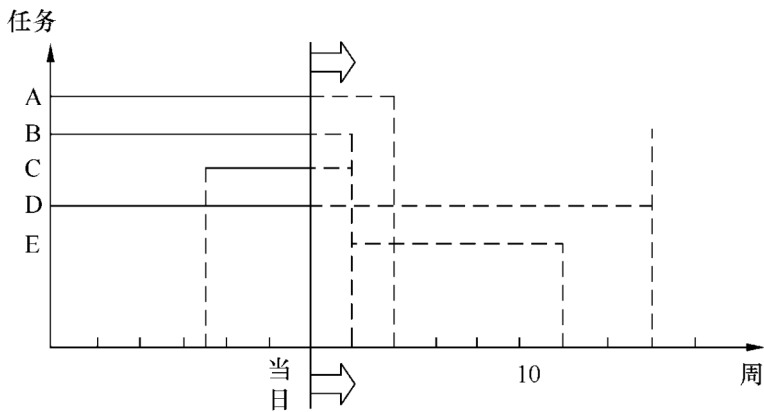
美国卡内基·梅隆大学软件工程研究所在 1994 年发表了“人员管理能力成熟度模型”(P-CMM)。该模型力图通过吸引、培养、激励、部署和聘用高水平的人才来提升软件组织的软件开发能力。

11.1. 进度管理

11.1.1. 甘特图

甘特图是表示工作进度计划以及工作实际进度情况最为简明的图示方法。

- 1. 横坐标表示时间，水平线段表示子任务的工作阶段，可以为其命名。
- 2. 线段的起点和终点分别对应着该项子任务的开工时间和完成时间
 - 计划时间为虚线

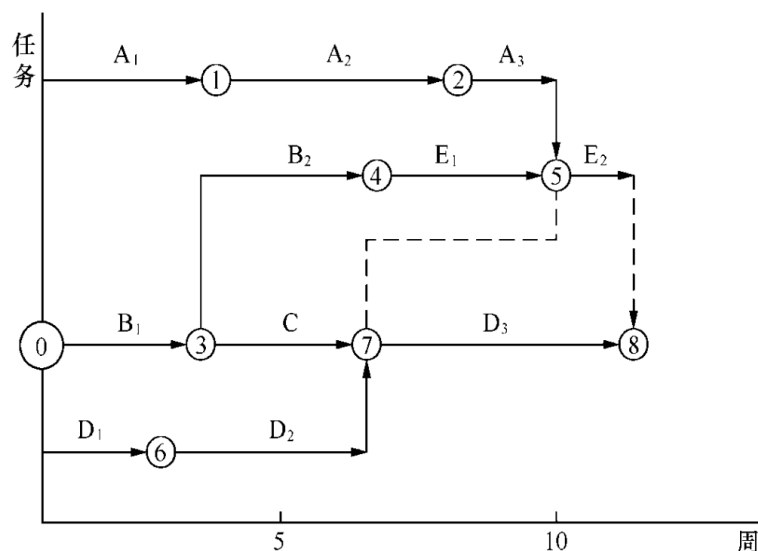


活 动		时间 (天)	责任人	开工日数													24	26
				2	4	6	8	10	12	14	16	18	20	22				
P1 详细设计	计划	5	张张	1	2	3												
	实际	6	张张	2	3	4	5											
P1 编程	计划	12	李李				7	8	9	10	11	12	13	14	15			
	实际	11	李李				8	9	10	11	12	13	14	15	16			
P1 单元测试	计划	6	李李												17	18	19	
	实际	3+	李李											18	19	20	21	
P2 详细设计	计划	3	张张				7	8										
	实际	3	张张				8	9	10									
P2 编程	计划	4	张王				9	10	11									
	实际	4	张王				10	11	12	13								
P2 单元测试	计划	2	张									15	16					
	实际		张									16						

甘特图可以清楚地表示各项子任务在时间对比上的关系，但无法表达多个子任务之间更为复杂的衔接关系。

11.1.2. 时标网状图

任务以有向线段表示，其指向点表示任务间的衔接点，并且都给予编号，可以显示出各子任务间的依赖关系。它显示出比甘特图具有优越性。



11.2. 配置管理

11.2.1. 配置管理概述

- 软件配置管理的目的是为某个过程或某个项目的软件项 建立和保持完整性，以便相关方使用。
- 软件配置管理要开展的活动包括：配置标识、配置控制、配置状态报告、配置评价以及发布管理、交付等。
- 软件配置管理的主要任务如下：
 1. 制订软件配置管理计划
 2. 实施变更管理
 3. 实施版本管理和发布管理

11.2.2. 软件配置标识

对配置项命名，命名需唯一性、可追溯性。个对象可用一组信息来唯一地标识，这组信息包括：（名字、描述、资源、实现）

对于一个基本对象来说，“实现”是指向“文本单元”的指针，而对于复合对象来说，则为 null（空）。

11.2.3. 变更管理

变更管理的任务：

1. 分析变更
2. 记录变更信息

3. 确保变更在受控条件下进行

配置数据库

基线、是软件生存期各开发阶段末尾的特定点，也被称为里程碑

11.2.4. 版本控制

版本管理是对系统不同版本进行标识和跟踪的过程。

系统发行是分配给客户一个版本，通常一次发行不仅只是提供一个可执行程序，或一套程序，可能还要包括：

- 配置文件：规定发行所作的特定安装；
- 数据文件：系统运行所需的数据；
- 安装程序：表明系统如何安装到目标机上；
- 电子文档或书面文档：这是对系统的描述。

版本控制系统：

- 管理代码历史版本
- 支持团队协作与分支管理

11.3. 质量管理

11.3.1. 正确理解和正确对待软件质量问题

软件质量：软件质量是指软件产品满足明确或隐含需求的能力。它不仅包括功能是否正确，还包括性能、可靠性、安全性、可用性、可维护性等多个方面。

解决软件质量问题的客观困难：抽象、复杂、多变、多样、多状态、个人因素、局限性和缺陷。

11.3.2. 质量管理措施

1. 明确质量管理的四个重要环节：质量策划、质量控制、质量保证、质量改进
2. 明确区分质量控制与质量保证：质量控制、质量保证的目的分别是——尽力使产品达到质量要求、为产品达到质量要求提供信任
3. 重视测试人员的配备、组织及测试工作
4. 强调过程的持续改进，使之实现制度化
5. 加强测试人员的质量意识和职业道德教育

11.4. 小结

- 软件项目管理中用于估计工作量的指标主要包括哪些？
- 常用的质量保障措施和方法有哪些？

软件项目管理中用于估计工作量的指标主要包括哪些？

工作量估算方法分为 2 类：

1. 直接估算：基于 WBS 直接估算人天工作量
2. 间接估算：先估算软件规模，再转换成人天工作量
 - 基于代码行
 - 基于功能点
 - 项目的工作量按项目将要投入的人工来考虑，以一个人工作一个月为单位，记为“人月”
 - $\text{工作量} = \text{规模} / \text{生产率}$
 - 生产率是平均每个人月完成的源程序行数，可记为 KLOC / 人月或 FP / 人月。

常用的质量保障措施和方法有哪些？

质量不仅依赖开发阶段，还需：

- 测试
- 代码审查
- 自动化测试
- 持续集成